

Robot Perception and Learning

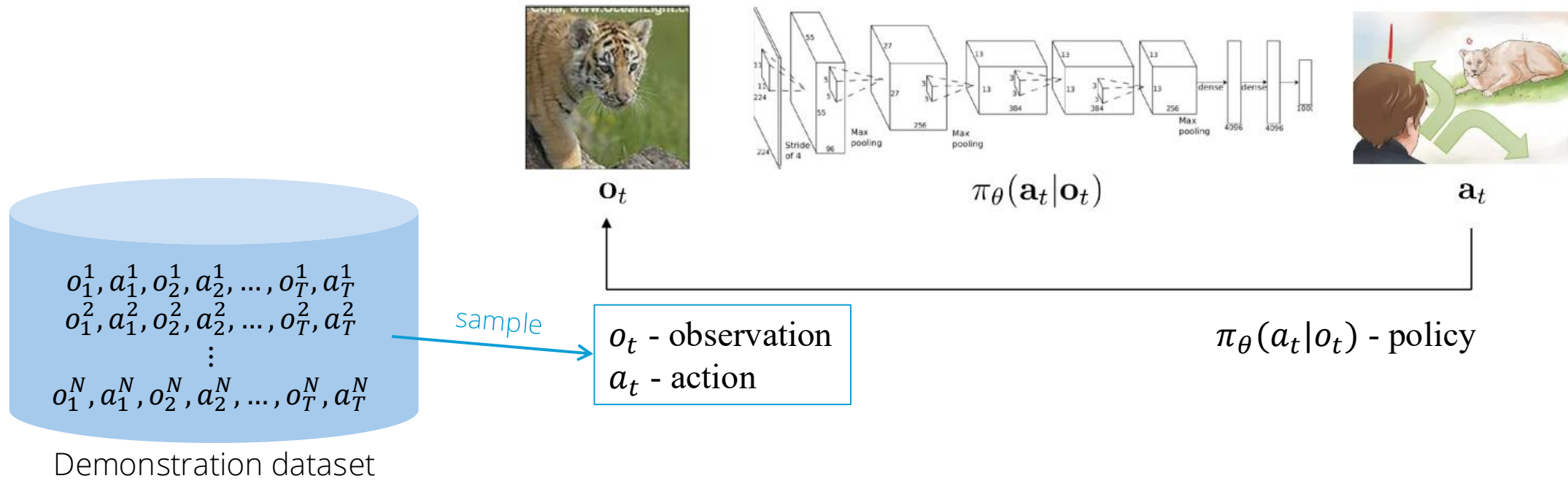
Markov Decision Processes, Policy Iteration, Dynamic Programming, Monte Carlo Method, Temporal Difference

Tsung-Wei Ke

Fall 2026

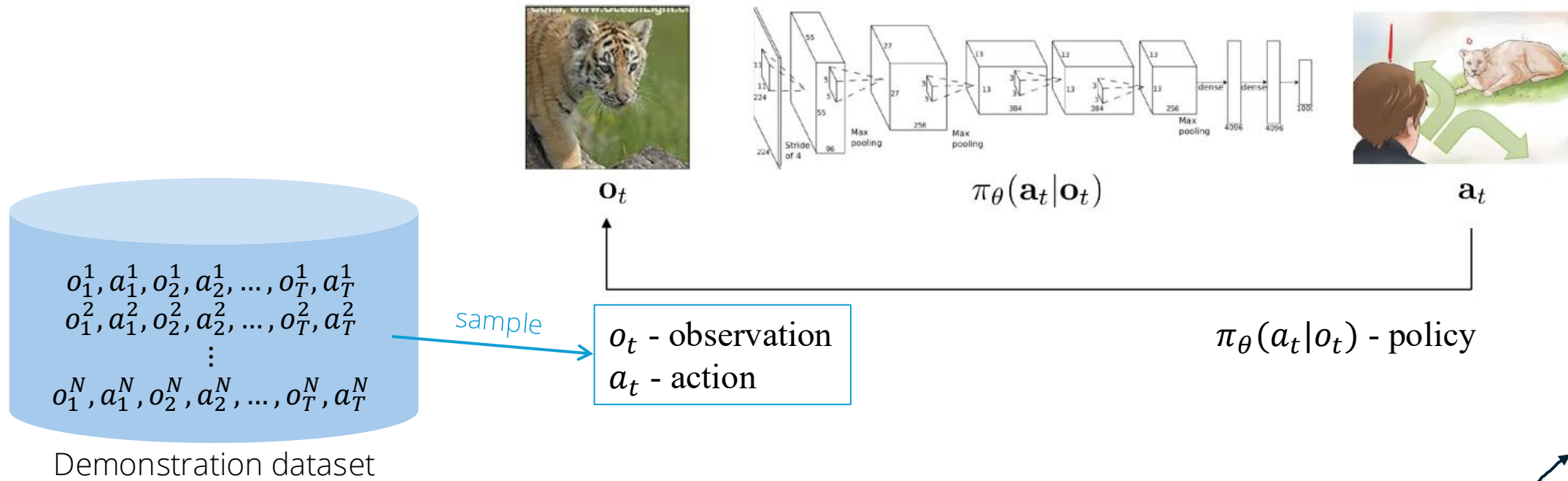


The Formulation of Imitation Learning is Limited

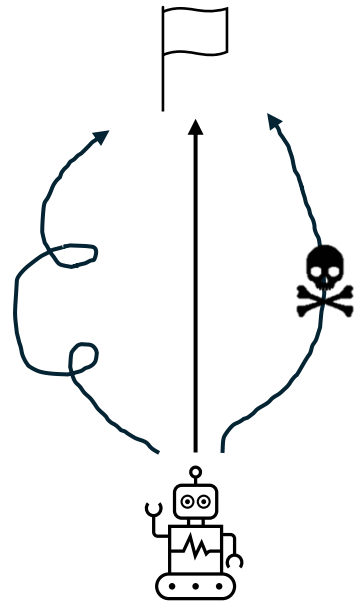


- Models learn only “the mapping” between observations and actions, instead of “the effect” of each action on an observation

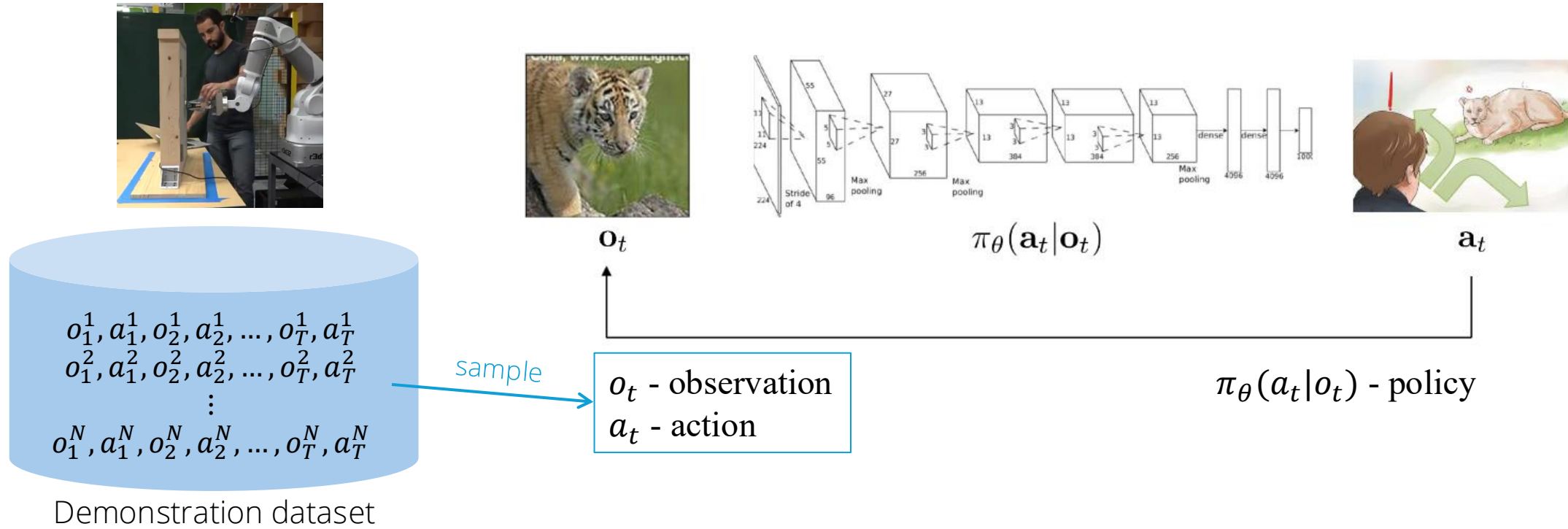
The Formulation of Imitation Learning is Limited



- Models learn only “the mapping” between observations and actions, instead of “the effect” of each action on an observation
 - Can we achieve the goal with the current action?
 - How long does it take to achieve the goal?
 - How will the world change?



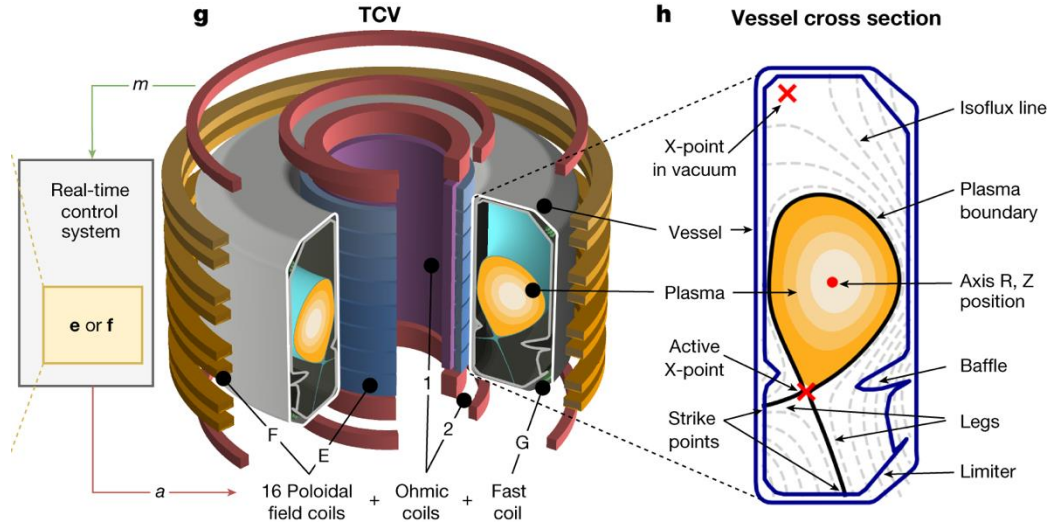
The Formulation of Imitation Learning is Limited



- Models learn only “the mapping” between observations and actions, instead of “the effect” of each action on an observation
- Imitation learning assumes the existence of an expert, limiting the application scenario

Can We Have a General Learning Framework for All Different Tasks?

Power Plant Control



Magnetic control of tokamak plasmas through deep reinforcement learning. Degraeve et al.

Trading

Market Summary > NVIDIA Corp

119.37 USD

+70.82 (145.87%) ↑ past year

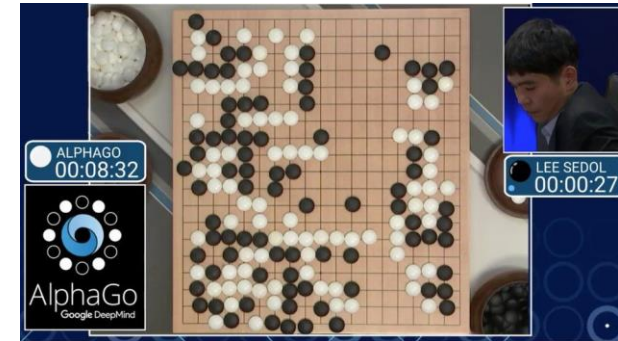
Closed: Aug 30, 7:59 PM EDT • Disclaimer
After hours 119.23 -0.14 (0.12%)

1D 5D 1M 6M YTD 1Y 5Y Max



Open	119.53	Mkt cap	2.93T	CDP score	B
High	121.75	P/E ratio	56.07	52-wk high	140.76
Low	117.22	Div yield	0.034%	52-wk low	39.23

Gaming



Adaptive VisuoMotor Control



AnyCar to Anywhere: Learning Universal Dynamics Model for Agile and Adaptive Mobility. Xiao et al.

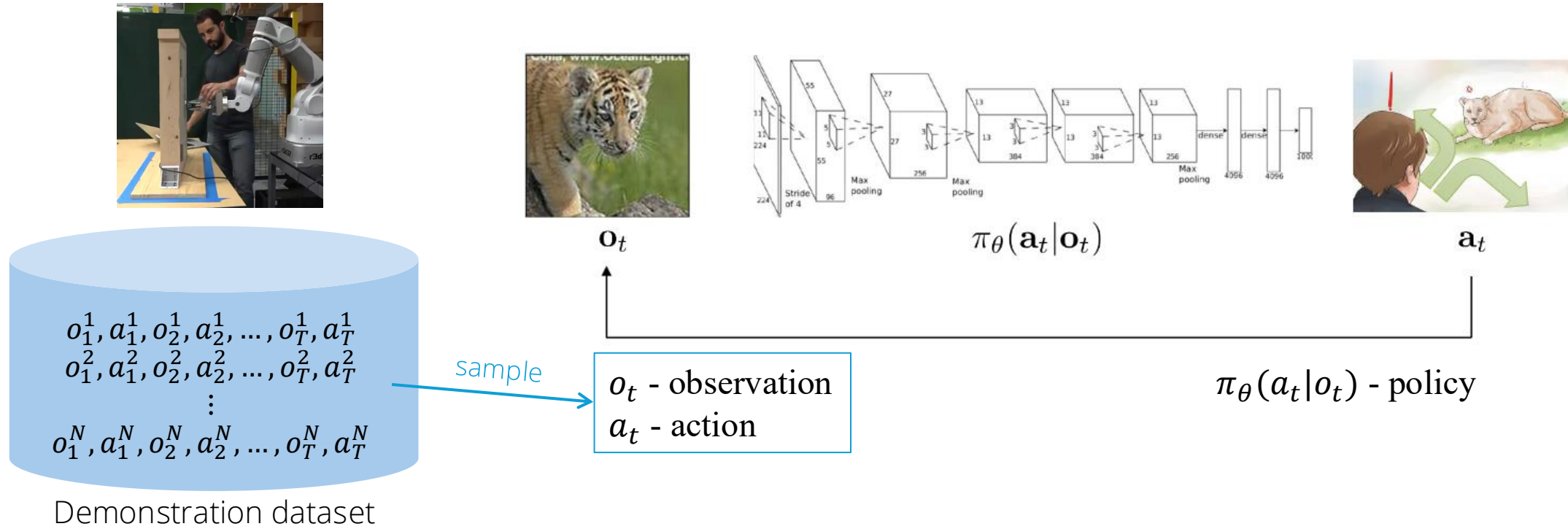
Content Generation



Training Animals

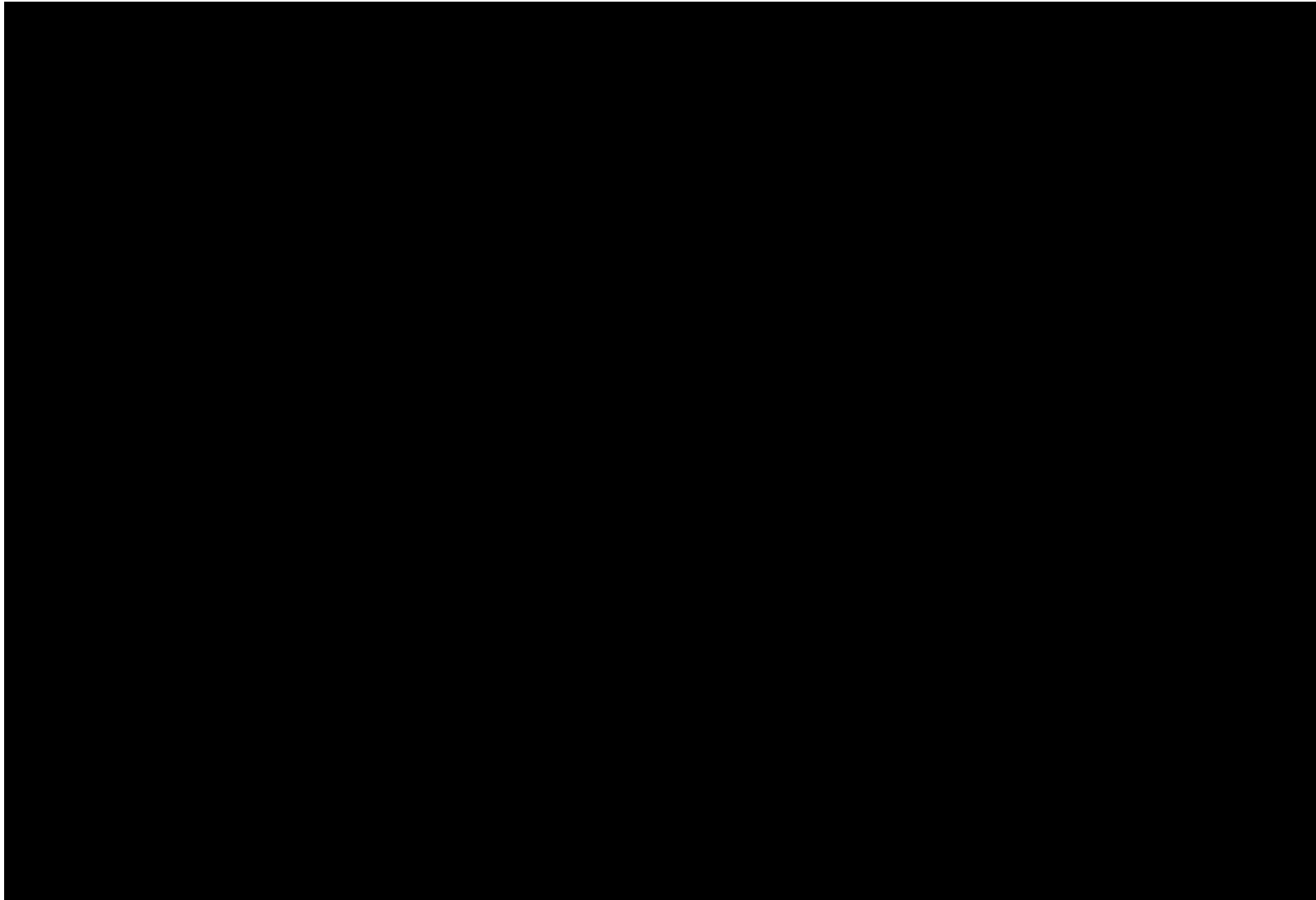


The Formulation of Imitation Learning is Limited



- Models learn only “the mapping” between observations and actions, instead of “the effect” of each action on an observation
- Imitation learning assumes the existence of an expert, limiting the application scenario
- Imitation learning is “at best” as good as the expert

Can We Reformulate the Learning Problem with Rewards not Action Labels?



Can We Reformulate the Learning Problem with Trial-and-Error not Imitation?



A Learning Framework that Solves Locomotion

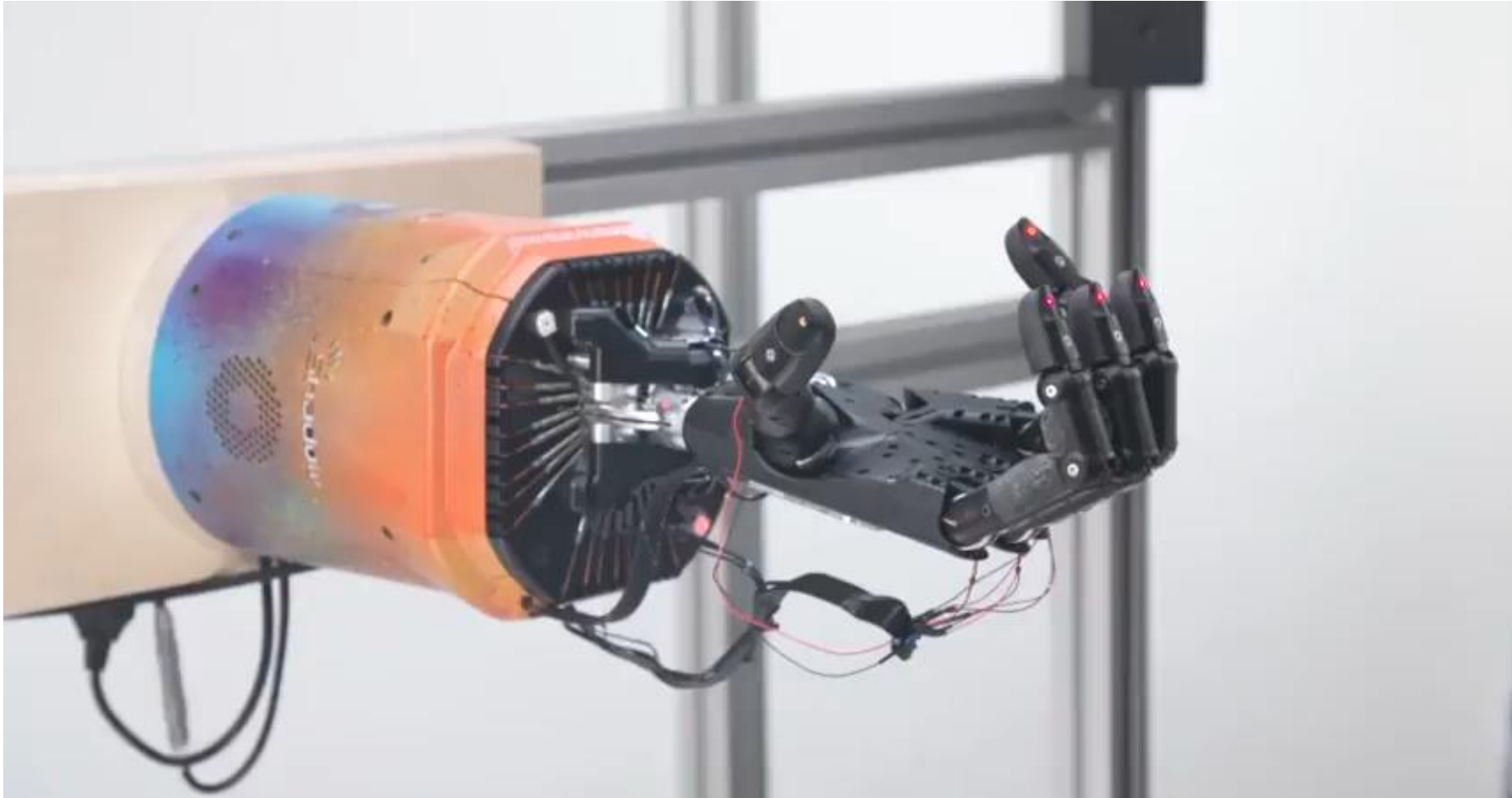


Rocky area next to river bed

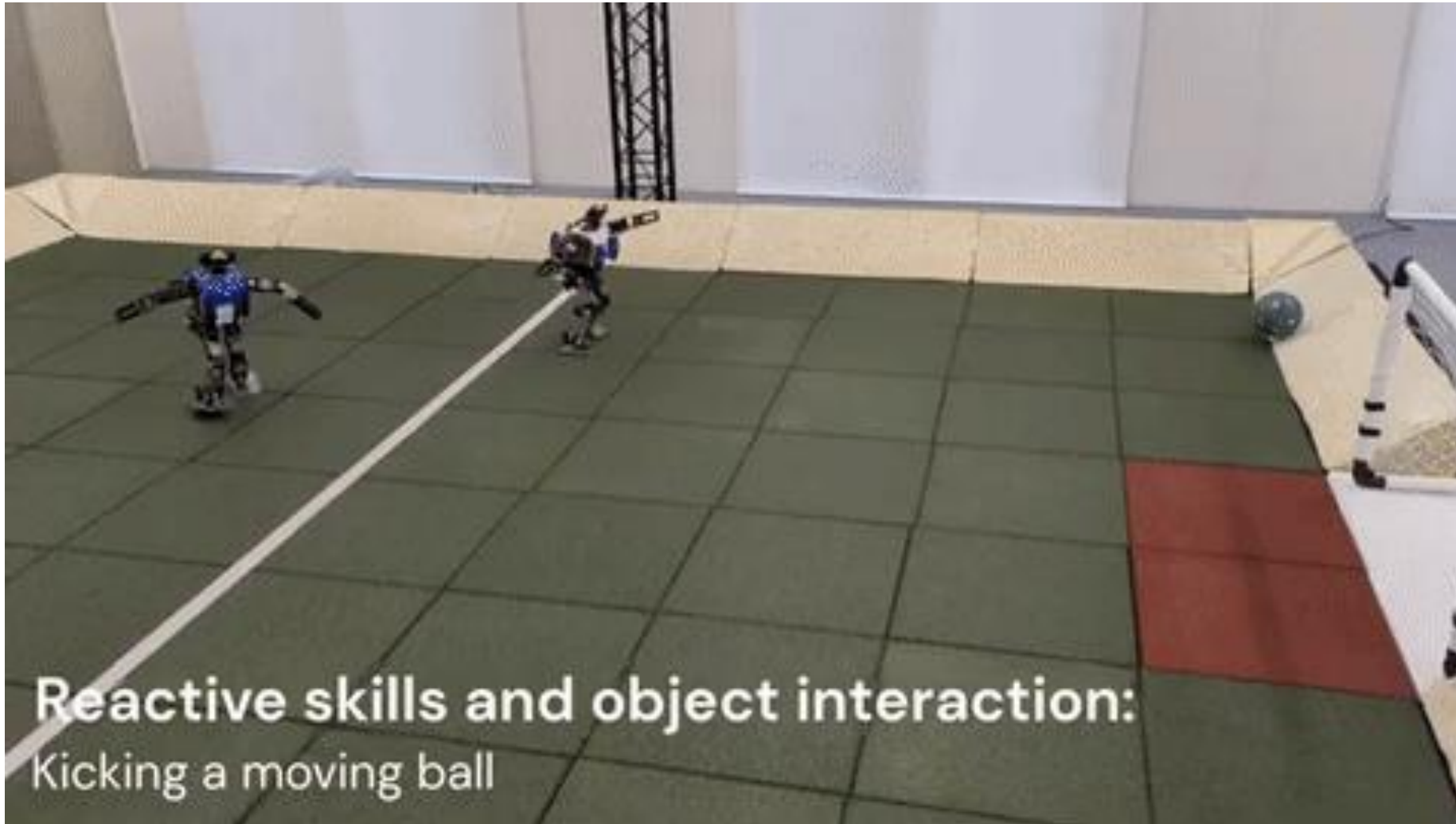
A Learning Framework that Solves Complex Locomotion



A Learning Framework that Solves Rubik's Cube



A Learning Framework that Solves Multi-Agent Competition



A Learning Framework that Surpasses Experts



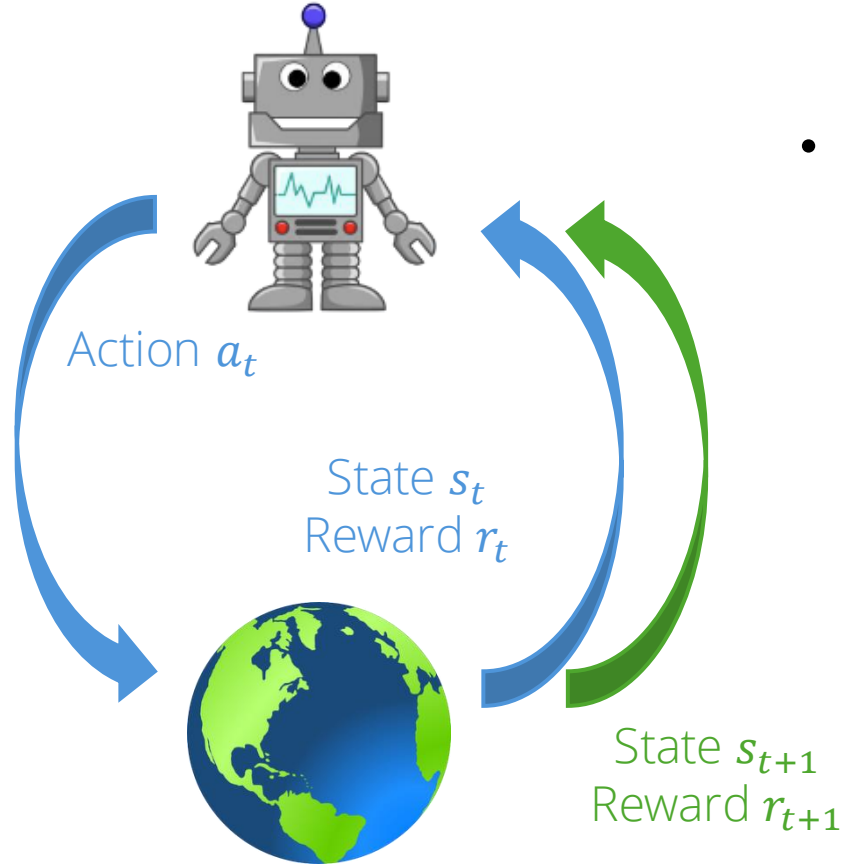
Content

- Formulation of reinforcement learning
- Multi-arm Bandit Problem
 - Exploration: Act randomly to explore information as much as possible
 - Exploitation: Act greedily to perform as best as possible
- Markovian Decision Making
 - Value estimation with Dynamic Programming
 - Policy Improvement
- Value estimation with Monte Carlo approach
- Value estimation with Temporal-Difference learning

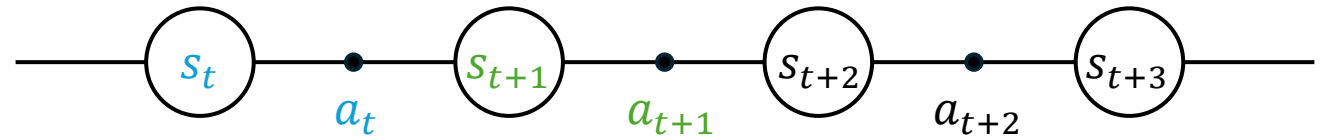
Content

- Formulation of reinforcement learning
- Multi-arm Bandit Problem
 - Exploration: Act randomly to explore information as much as possible
 - Exploitation: Act greedily to perform as best as possible
- Markovian Decision Making
 - Value estimation with Dynamic Programming
 - Policy Improvement
- Value estimation with Monte Carlo approach
- Value estimation with Temporal-Difference learning

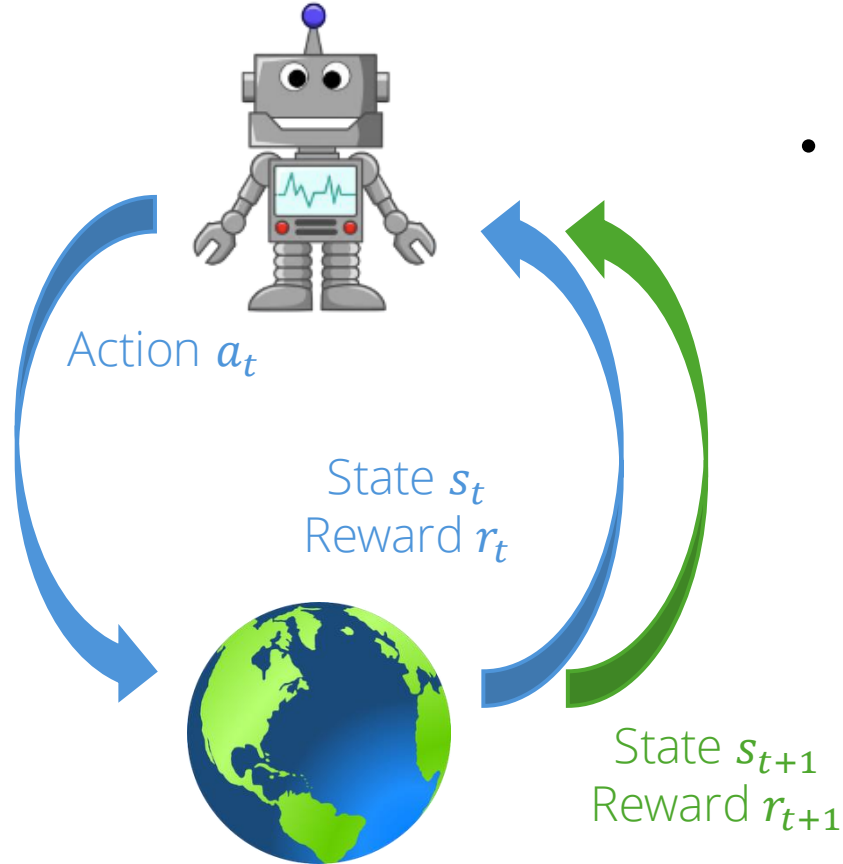
Actions Induce Changes of the Environment



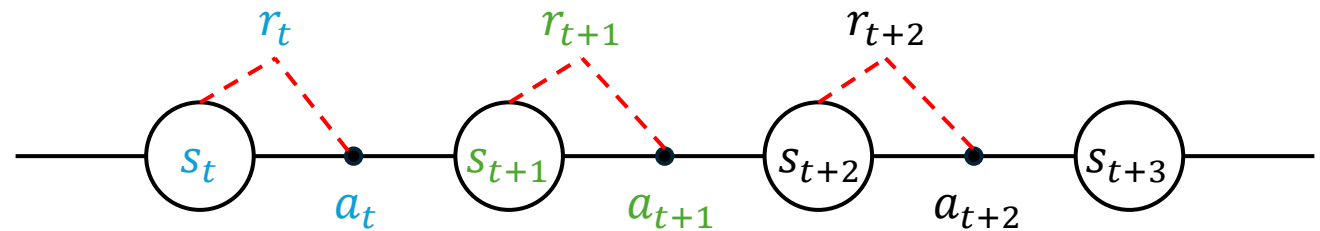
- A trajectory of interaction in the environment



The Reward Function $r(s, a)$ Estimates Goodness of Each State-Action Pair



- A trajectory of interaction in the environment



high reward



low reward



? reward

In fact, Defining Reward Functions is Tricky...

Success ✓



Fail ✗

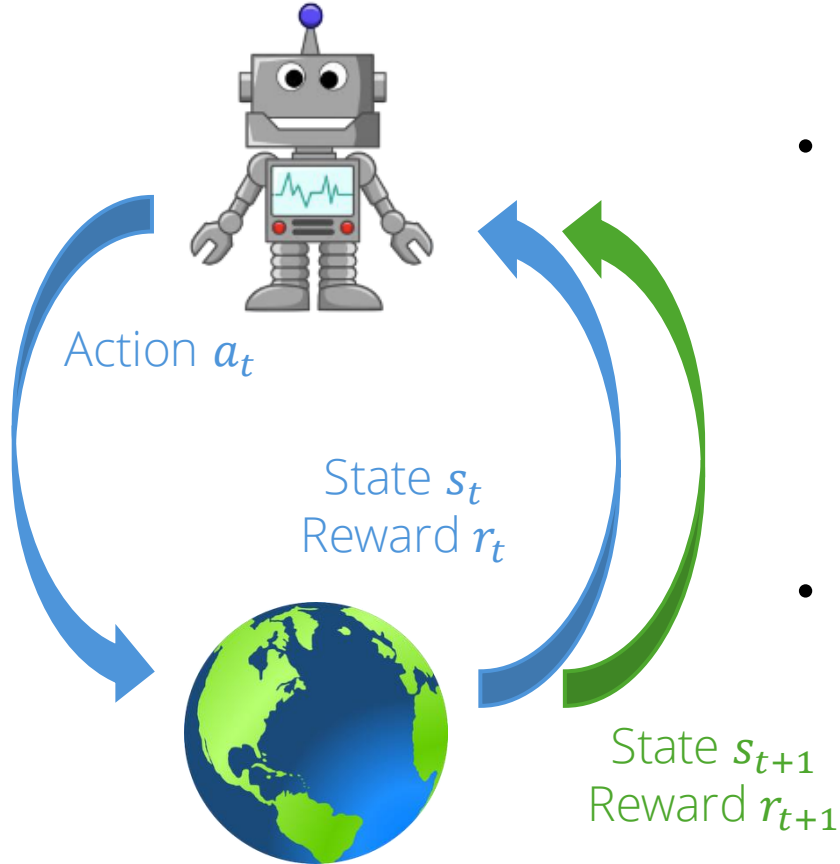


Leading others ?
Energy efficiency ?
Tire wear ?
Safety ?
Acceleration smoothness ?
Circular motion ?

- The most general form of rewards
 - Don't need domain knowledge
- The sparsest form of rewards
 - Credit assignment issue
- Learning with only these rewards is sample inefficient

- The biased form of rewards
 - Need domain knowledge
- Denser forms of rewards
 - Guide / facilitate learning with expert knowledge / prior
- Require huge human efforts
 - LLMs sometimes can help

Reinforcement Learning: Learns to Maximize the Total Reward of an Episode of Interaction



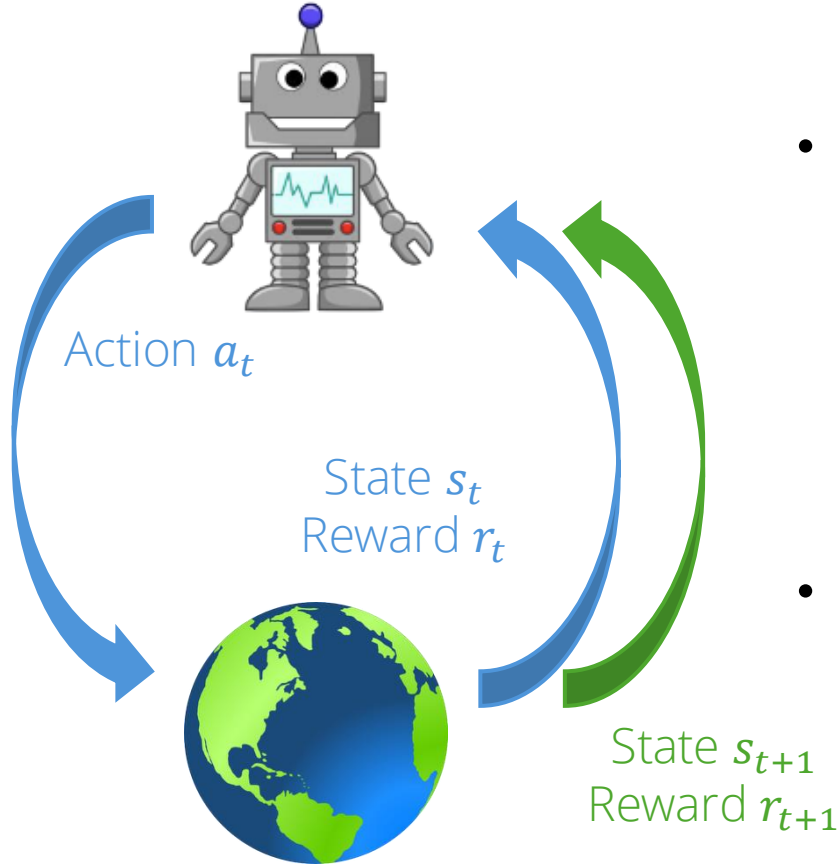
- A trajectory of interaction in the environment

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

- Maximize the expected value of the cumulative sum of reward

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Reinforcement Learning: Learns to Maximize the Total Reward of an Episode of Interaction



- A trajectory of interaction in the environment

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

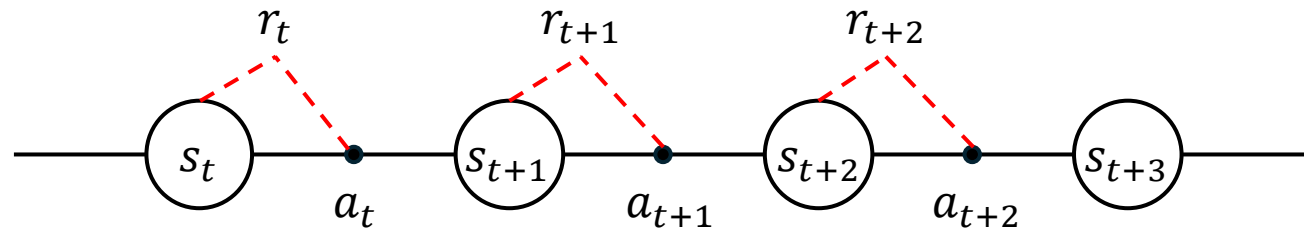
Previous actions decide what future states we will perceive

- Maximize the expected value of the cumulative sum of reward

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

A Sequence of Actions are Not Temporally Independent...

- An action induce changes of the environment state
 - An action may have low reward now, but lead to very high-reward future states
 - An action may have high reward now, but lead to very low-reward future states

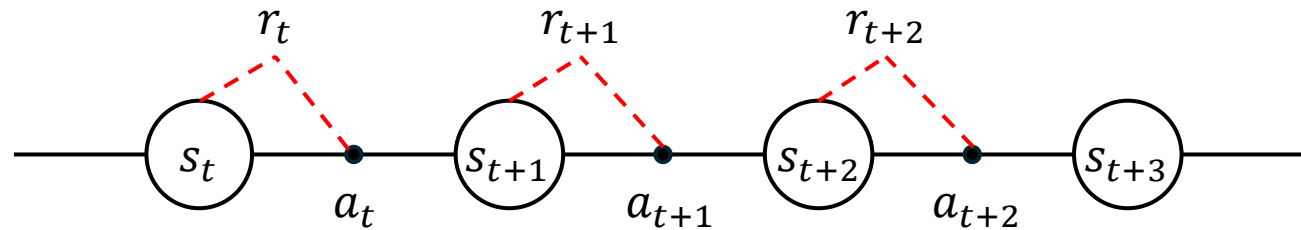


$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Consider an entire episode of the interaction, not each individual action

A Sequence of Actions are Not Temporally Independent...

- An action induce changes of the environment state
 - An action may have low reward now, but lead to very high-reward future states
 - An action may have high reward now, but lead to very low-reward future states



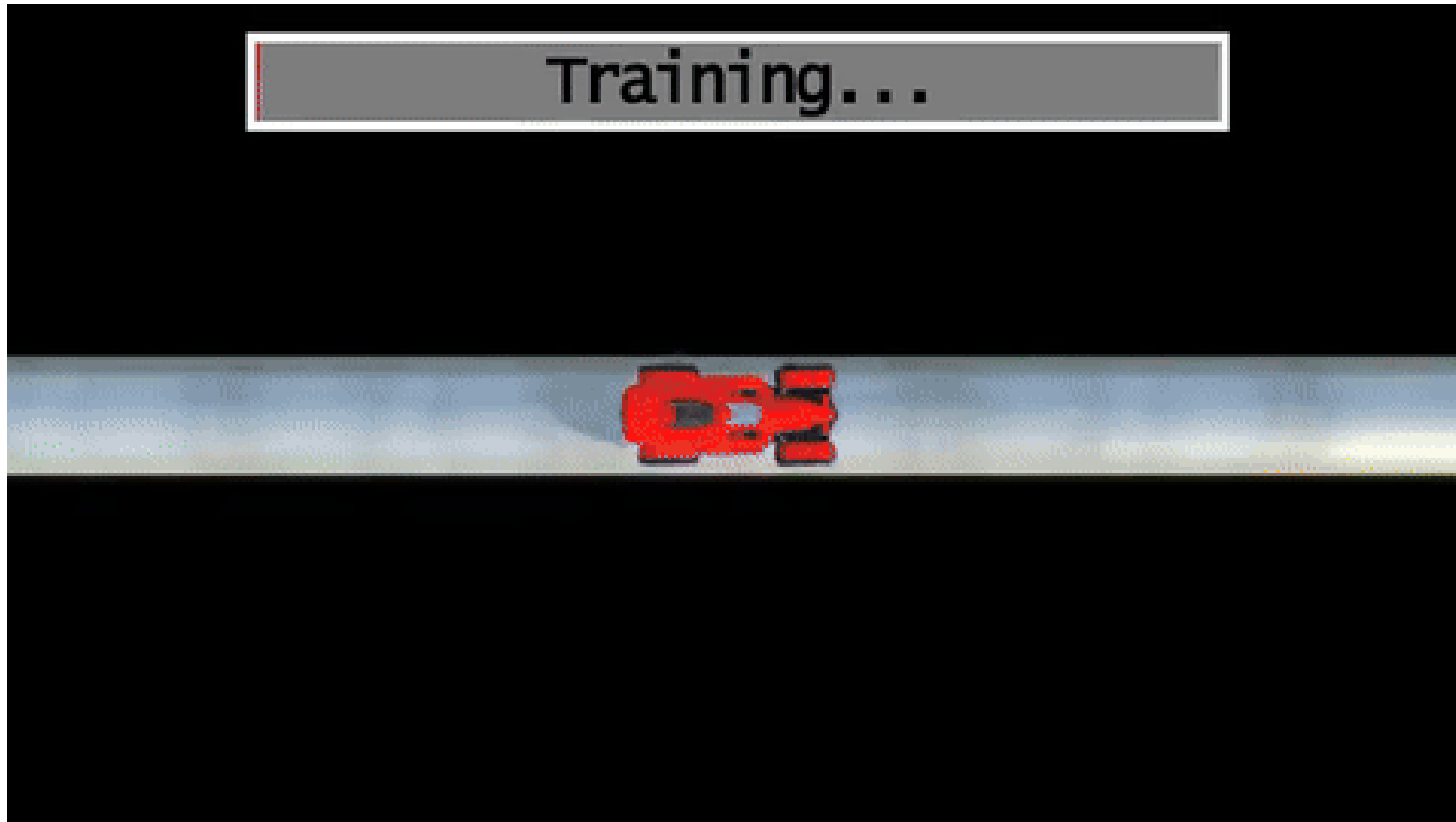
Or, we can have better estimation of state-action reward, that consider possible future rewards?

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, \mathbf{a}_t) \right]$$

The equation shows the optimal policy θ^* is the one that maximizes the expected return. The expectation is taken over the entire episode τ , which is sampled from the policy $p_{\theta}(\tau)$. The return is the sum of rewards $r(s_t, \mathbf{a}_t)$ over time t . A blue box highlights $\tau \sim p_{\theta}(\tau)$ with a blue arrow pointing down to the text below. A green box highlights the sum of rewards $\left[\sum_t r(s_t, \mathbf{a}_t) \right]$ with a green arrow pointing up to the text above.

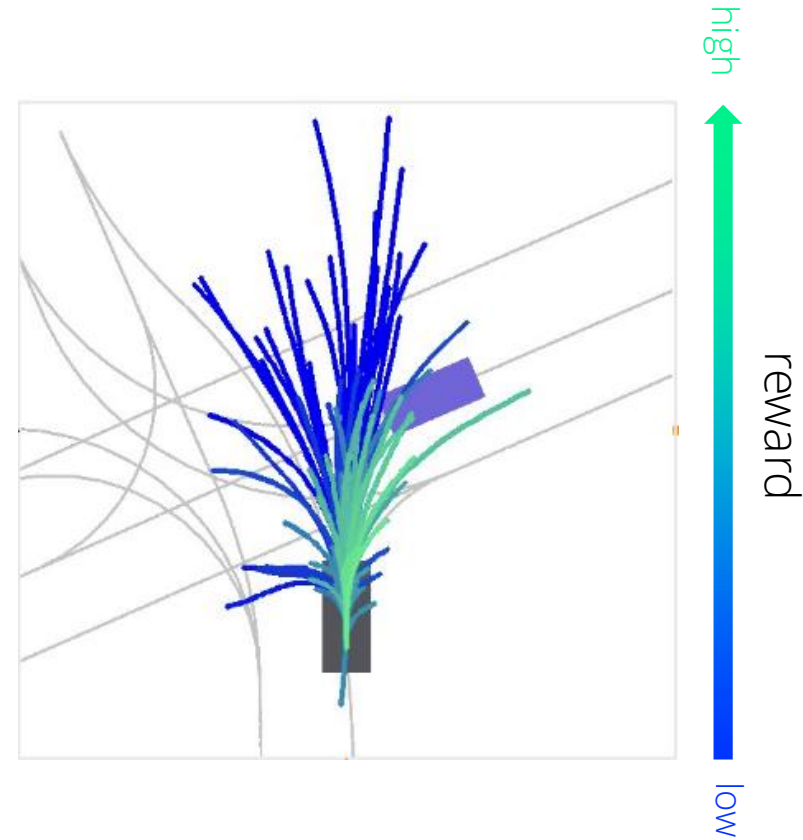
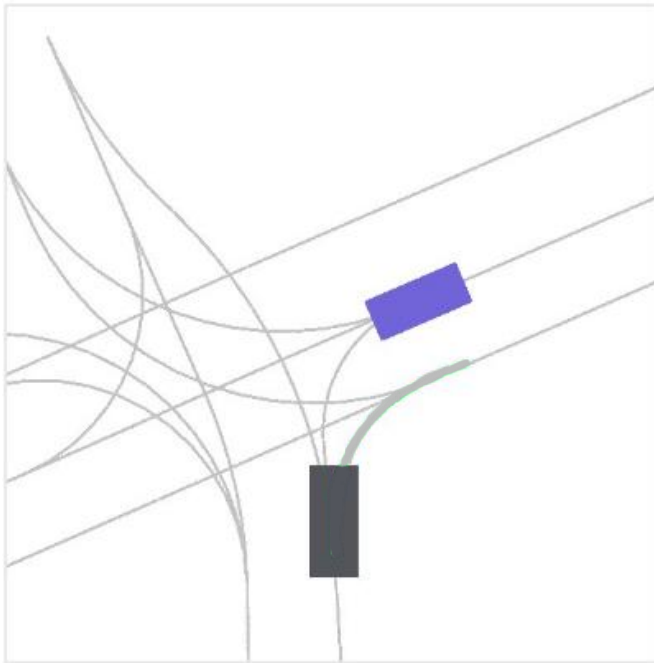
Consider an entire episode of the interaction, not each individual action

Reinforcement Learning: Trial-and-Error Learning



Reinforcement Learning: Trial-and-Error Learning

Task: make a right turn



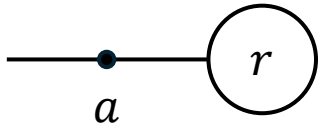
- Explore: discover actions that may have high rewards, but usually are suboptimal
- Exploit: take the current best action

Content

- Formulation of reinforcement learning
- Multi-arm Bandit Problem
 - Exploration: Act randomly to explore information as much as possible
 - Exploitation: Act greedily to perform as best as possible
- Markovian Decision Making
 - Value estimation with Dynamic Programming
 - Policy Improvement
- Value estimation with Monte Carlo approach
- Value estimation with Temporal-Difference learning

Let's Consider a Non-Sequential Setup

- Non-sequential setup:
 - Each action results in an **immediate reward**
 - We want to choose actions that maximize our immediate reward in expectation
 - There is no state
- This simplified setup is also called “Bandit Problem”
- This simplified setup helps us to focus on key components / tools in RL



Let's Consider a Non-Sequential Setup

- Non-sequential setup:
 - Each action results in an **immediate reward**
 - We want to choose actions that maximize our immediate reward in expectation
 - There is no state
- For example, choosing which restaurant to go
 - **Actions:** the restaurants to choose from
 - **Rewards:** your happiness
- Let's say you can eat outside 100 times, what's the best strategy to maximize your total happiness?
 - **Explore:** discover new restaurants
 - **Exploit:** go to the favorite restaurant
- Without exploration, you may end up going to 7-11 all the time; Without exploitation, you keep trying new but bad restaurants.



Let's Consider a Non-Sequential Setup

- Non-sequential setup:
 - Each action results in an **immediate reward**
 - We want to choose actions that maximize our immediate reward in expectation
 - There is no state
- For example, choosing which restaurant to go
 - **Actions:** the restaurants to choose from
 - **Rewards:** your happiness
- Let's say you can eat outside 100 times, what's the best strategy to maximize your total happiness?
 - **Explore:** discover new restaurants
 - **Exploit:** go to the favorite restaurant

This is the key question in RL

- Without exploration, you may end up going to 7-11 all the time; Without exploitation, you keep trying new but bad restaurants.



Multi-Armed Bandit Problem

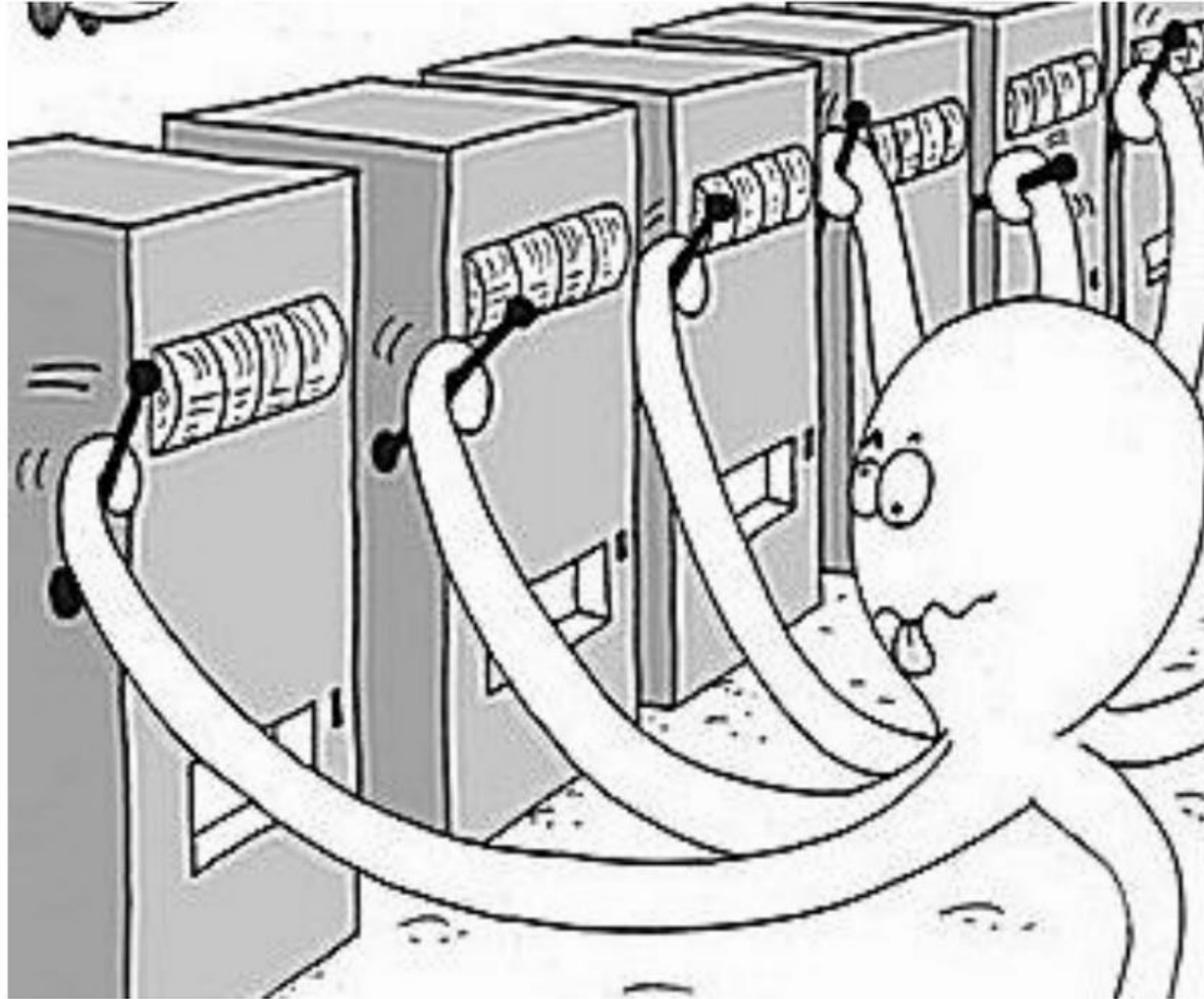


Image credit Microsoft

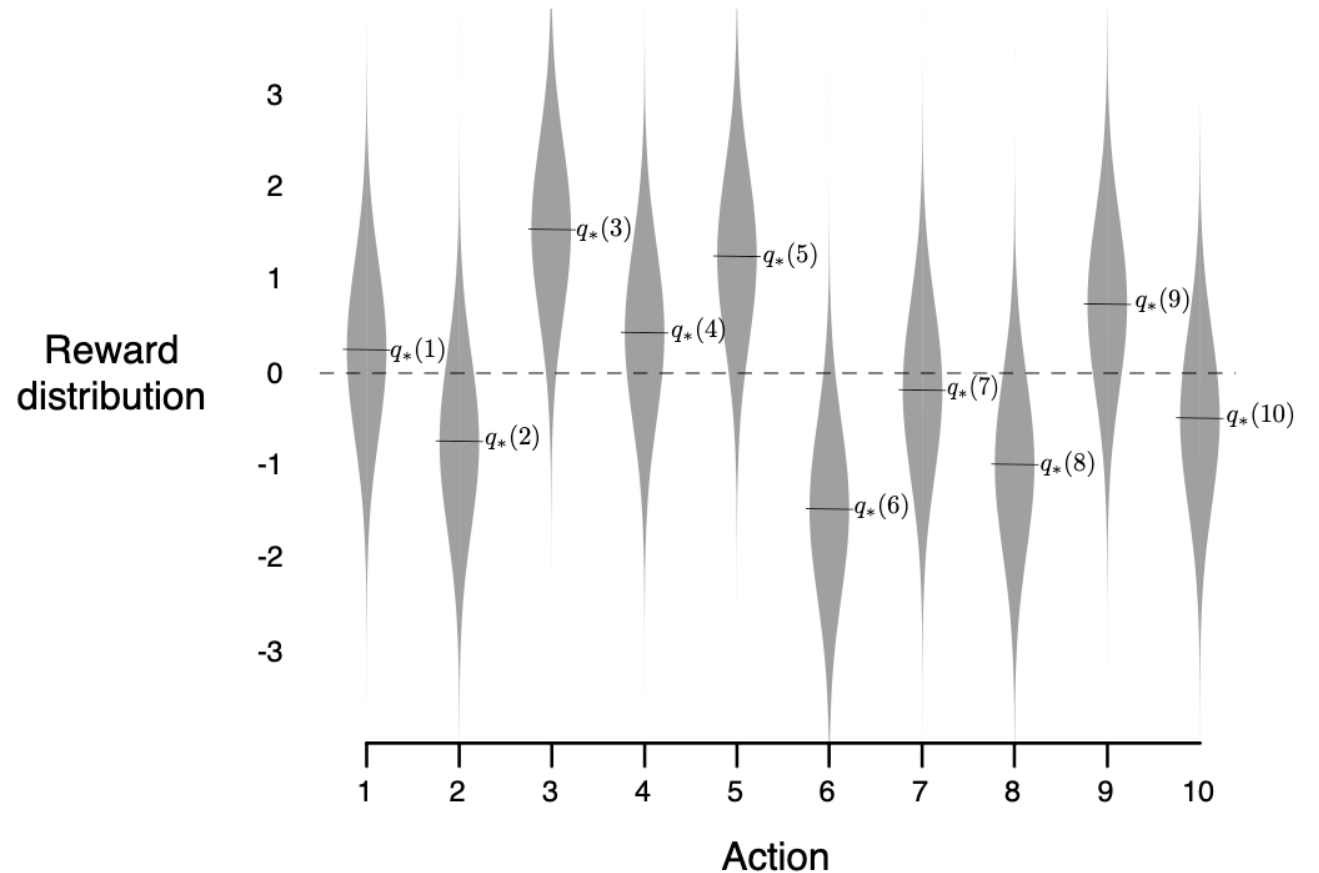
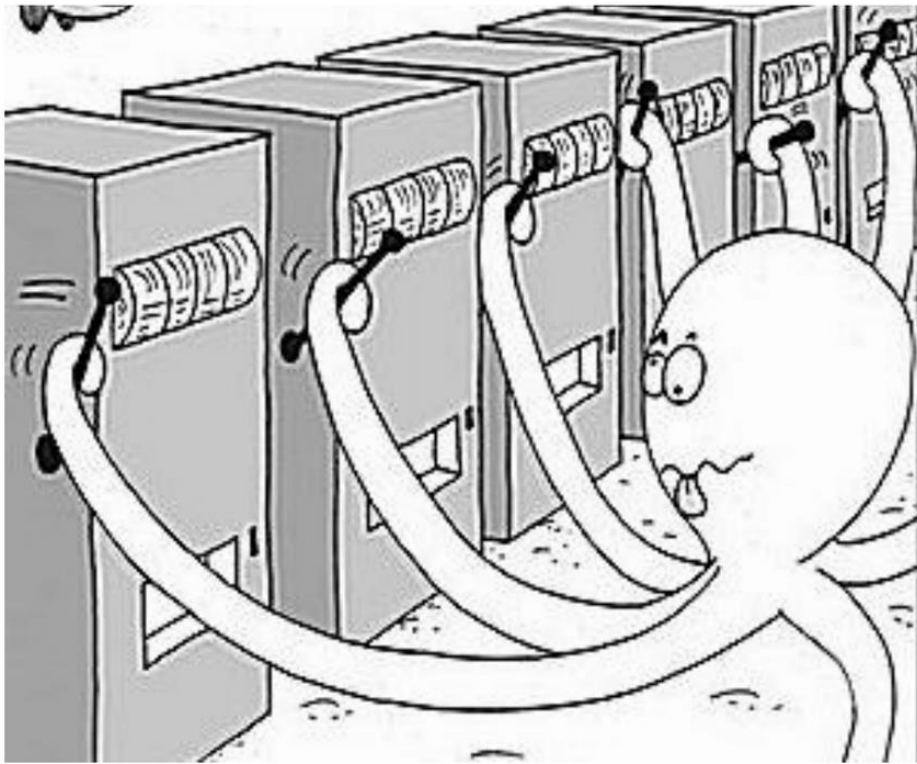
Multi-Armed Bandit Problem



$$\begin{array}{ccc} r_1 \sim P_1 & r_2 \sim P_2 & r_3 \sim P_3 \\ P_1 = \mathcal{N}(\mu_1, \sigma_1) & P_2 = \mathcal{N}(\mu_2, \sigma_2) & P_3 = \mathcal{N}(\mu_3, \sigma_3) \end{array}$$

- At timestep t
 - The agent plays one of the K arms
 - The k^{th} arm produces reward $r_{k,t}$ when played
 - The reward $r_{k,t}$ is drawn from a probability distribution P_k with mean μ_k and std σ_k (Note: the distribution could be non-Gaussian).
 - The real reward distribution is unknown
- How to maximize the total reward for playing the bandit machines within a finite or infinite horizon?

What's the Strategy if the Reward Distribution is Known



If the distribution is known, the best strategy is to **exploit**: play the arm with the highest expected reward

How to Formulate the Behavior of Exploitation?

- Expected reward: $q^*(a_k) = \mathbb{E}[r_t | A_t = a_k]$
- Action-value estimates: $Q_t(a_k)$ ← Characterizes how good an action a_k is

Check Appendix 1 for incremental implementation of value estimation

How to Formulate the Behavior of Exploitation?

- Expected reward: $q^*(a_k) = \mathbb{E}[r_t | A_t = a_k]$
- Action-value estimates: $Q_t(a_k)$ ← Characterizes how good an action a_k is
- Greedy action selection method: select the action with the highest estimated value:

$$A_t^* = \arg \max_a Q_t(a)$$

- If $A_t = A_t^*$, you are *exploiting* your current knowledge of the values of the actions
- If $A_t \neq A_t^*$, you are *exploring*. You improve your estimate of the non-greedy actions

How to Formulate the Behavior of Exploitation?

- Expected reward: $q^*(a_k) = \mathbb{E}[r_t | A_t = a_k]$
- Action-value estimates: $Q_t(a_k)$ ← Characterizes how good an action a_k is
- Greedy action selection method: select the action with the highest estimated value:

$$A_t^* = \arg \max_a Q_t(a)$$

- If $A_t = A_t^*$, you are *exploiting* your current knowledge of the values of the actions
- If $A_t \neq A_t^*$, you are *exploring*. You improve your estimate of the non-greedy actions
- If $Q_t(a_k) \approx q^*(a_k)$, no need for exploration. Greedy action selection is the best strategy.
- If $Q_t(a_k) \neq q^*(a_k)$, you need exploring different actions to refine your action-value estimation, otherwise, you end up choosing sub-optimal actions

The Exploration / Exploitation Dilemma

- Exploitation: Make the best decision given current information
- Exploration: Sacrifice some action budget to gather more information
- **Key question: when to explore and when to exploit?**
- Idea 1: ϵ -greedy method, a naïve solution that decide to explore / exploit by throwing a dice with probability ϵ

ϵ -Greedy Method

A simple bandit algorithm

Initialize, for $a = 1$ to k :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0$$

Loop forever:

Explore or Exploit?

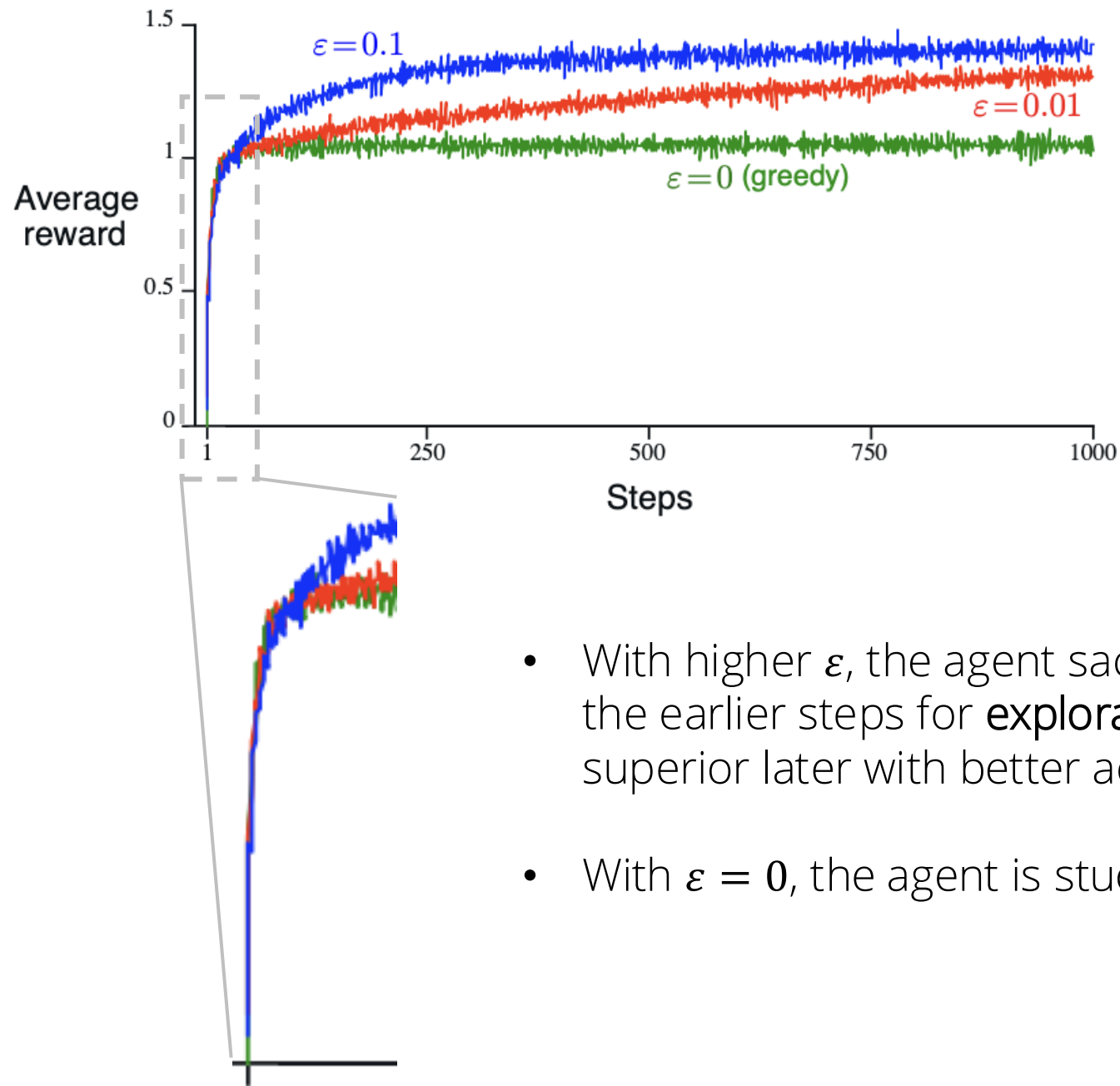
$$A \leftarrow \begin{cases} \operatorname{argmax}_a Q(a) & \text{with probability } 1 - \epsilon \quad (\text{breaking ties randomly}) \\ \text{a random action} & \text{with probability } \epsilon \end{cases}$$

$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

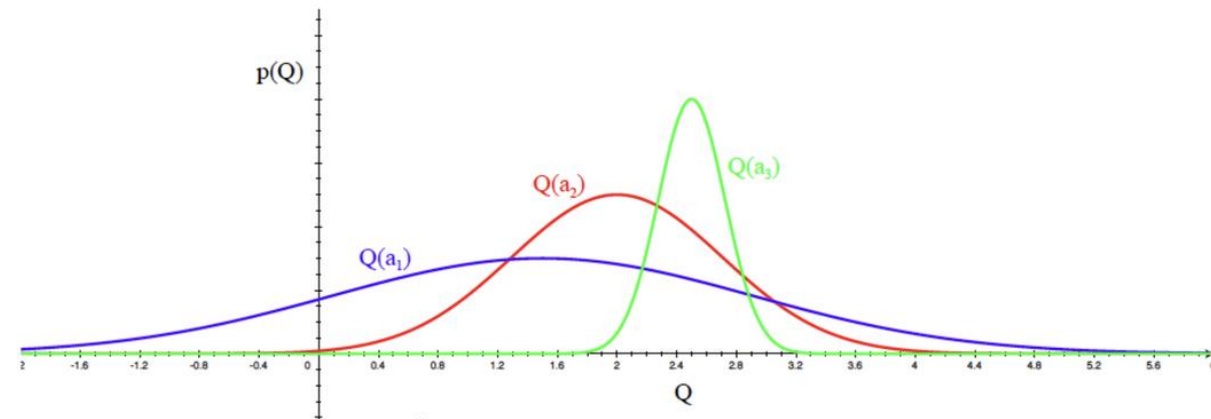
Update the action-value estimation..



- With higher ϵ , the agent sacrifice performance in the earlier steps for **exploration**, but it becomes superior later with better action-value estimation
- With $\epsilon = 0$, the agent is stuck in the local minimum

The Exploration / Exploitation Dilemma

- Exploitation: Make the best decision given current information
- Exploration: Sacrifice some action budget to gather more information
- Key question: when to explore and when to exploit?
- Idea 1: ϵ -greedy method, a naïve solution that decide to explore / exploit by throwing a dice with probability ϵ
- Idea 2: upper confidence bound (UCB) method, the more **uncertain** we are about an action-value, the more important it is to **explore** that action



Upper Confidence Bounds

- Estimate an **upper confidence** $U_t(a)$ for each action value such that with high probability:

$$q_*(a) \leq Q_t(a) + U_t(a)$$

Estimated mean

Estimated Upper
Confidence

- This upper confidence depends on the number of times action a has been selected
 - Small $N_t(a) \Rightarrow$ large $U_t(a)$ (estimated value is uncertain)
 - Large $N_t(a) \Rightarrow$ small $U_t(a)$ (estimated value is accurate)
- Select action maximizing **Upper Confidence Bound** (UCB)

$$a_t = \operatorname{argmax}_{a \in \mathcal{A}} Q_t(a) + U_t(a)$$

Upper Confidence Bound (UCB)

- A clever way of reducing exploration over time
- Estimate an upper bound on the true action values
- Select the action with the largest (estimated) upper bound

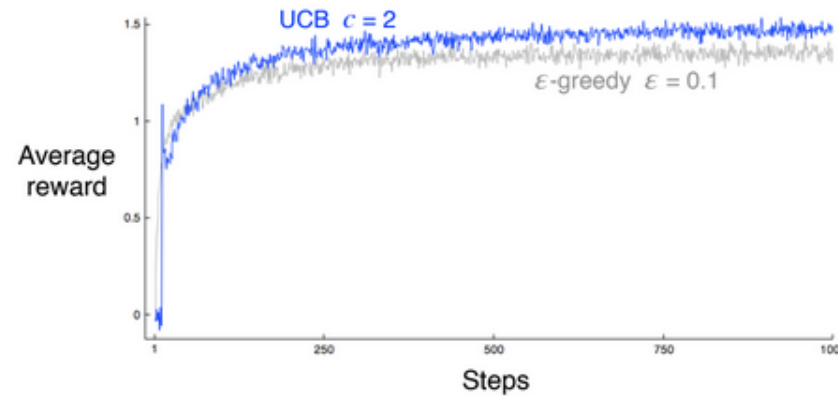
$$A_t \doteq \arg\max_a \left[Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$

- c is a hyper-parameter that trades-off explore/exploit
- the confidence bound grows with the total number of actions we have taken t but shrinks with the number of times we have tried this particular action $N_t(a)$. This ensures each action is tried infinitely often but still balances exploration and exploitation.
 - t : how many times I have played any action,
 - $N_t(a)$: how many times I have played action a in t interactions

Upper Confidence Bound (UCB)

- A clever way of reducing exploration over time
- Estimate an upper bound on the true action values
- Select the action with the largest (estimated) upper bound

$$A_t \doteq \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$



The Exploration / Exploitation Dilemma

- Exploitation: Make the best decision given current information
- Exploration: Sacrifice some action budget to gather more information
- Key question: when to explore and when to exploit?
- Idea 1: ϵ -greedy method, a naïve solution that decide to explore / exploit by throwing a dice with probability ϵ
- Idea 2: upper confidence bound (UCB) method, the more **uncertain** we are about an action-value, the more important it is to **explore** that action
- Idea 3: gradient bandit method, sample an action based on relative **preference** over other actions

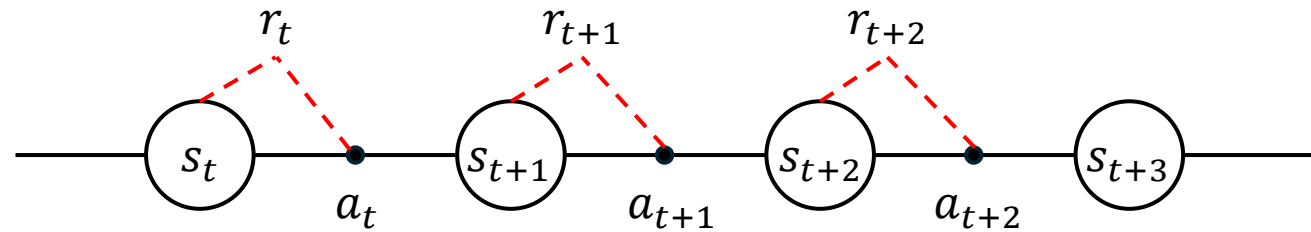
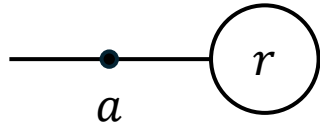
Check Appendix 2!

Content

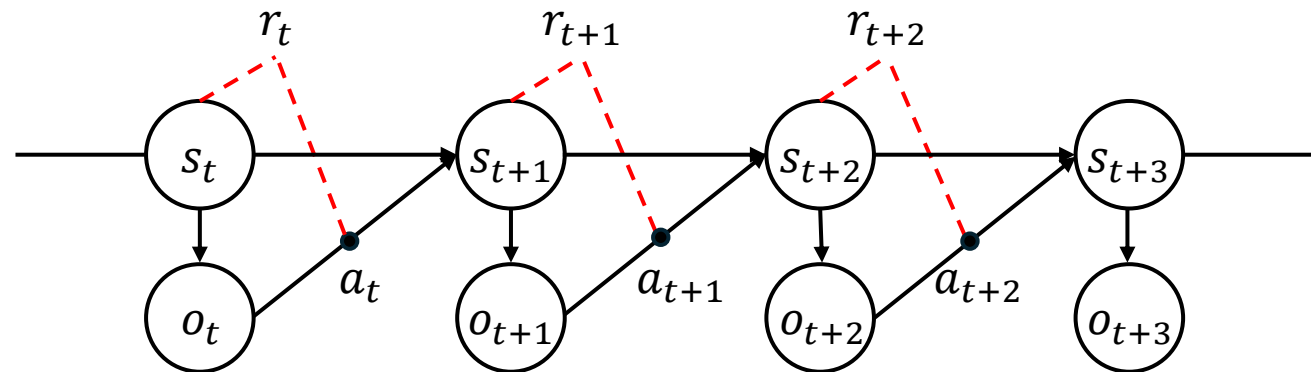
- Formulation of reinforcement learning
- Multi-arm Bandit Problem
 - Exploration: Act randomly to explore information as much as possible
 - Exploitation: Act greedily to perform as best as possible
- Markovian Decision Making
 - Value estimation with Dynamic Programming
 - Policy Improvement
- Value estimation with Monte Carlo approach
- Value estimation with Temporal-Difference learning

So Far, We've Considered a Non-Sequential Setup

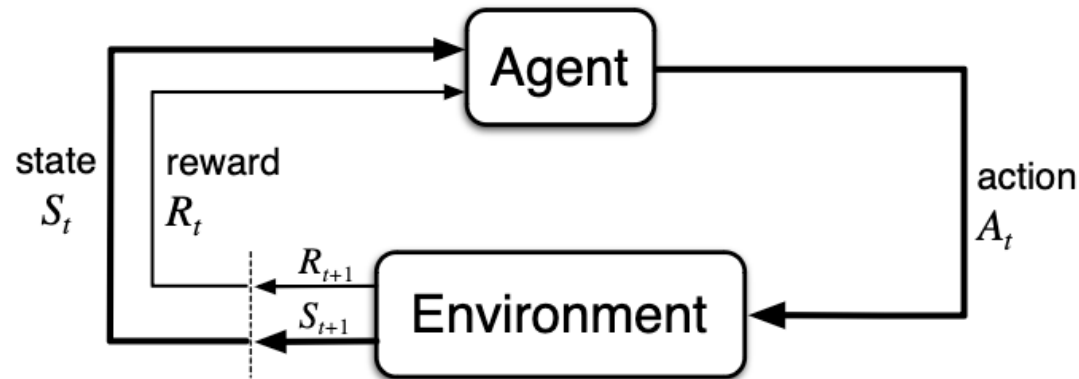
But in most cases, we have "states" and every action has sequential effect!



Moreover, we often don't know "states" but have only "observations"...



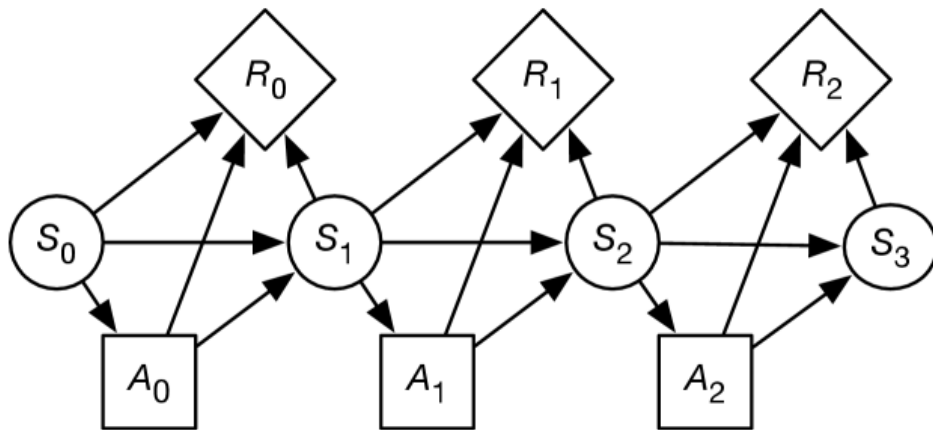
Let's First Define the Following Components



- **Agent:** the decision maker that senses the environment and decide “what actions” to take in the environment
- **Environment:** the world
- **Policy:** a mapping function from states / observations to actions.
- **Reward:** the signal indicates “**what**” you want a robot to achieve, not “**how**” you want it to achieve
- **State:** the representations that retain all “essential” information for decision making
- **(World) Model:** the transition function that maps states / observations and actions to future states / observations

MDP vs. POMDP

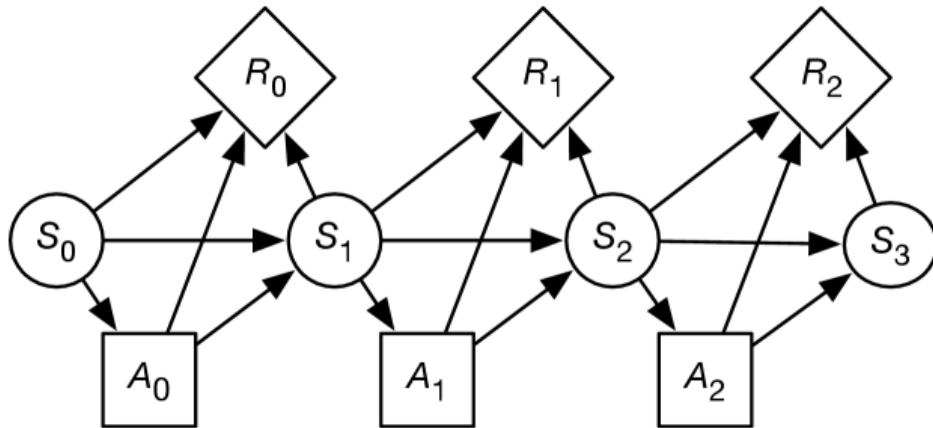
Markov Decision Process



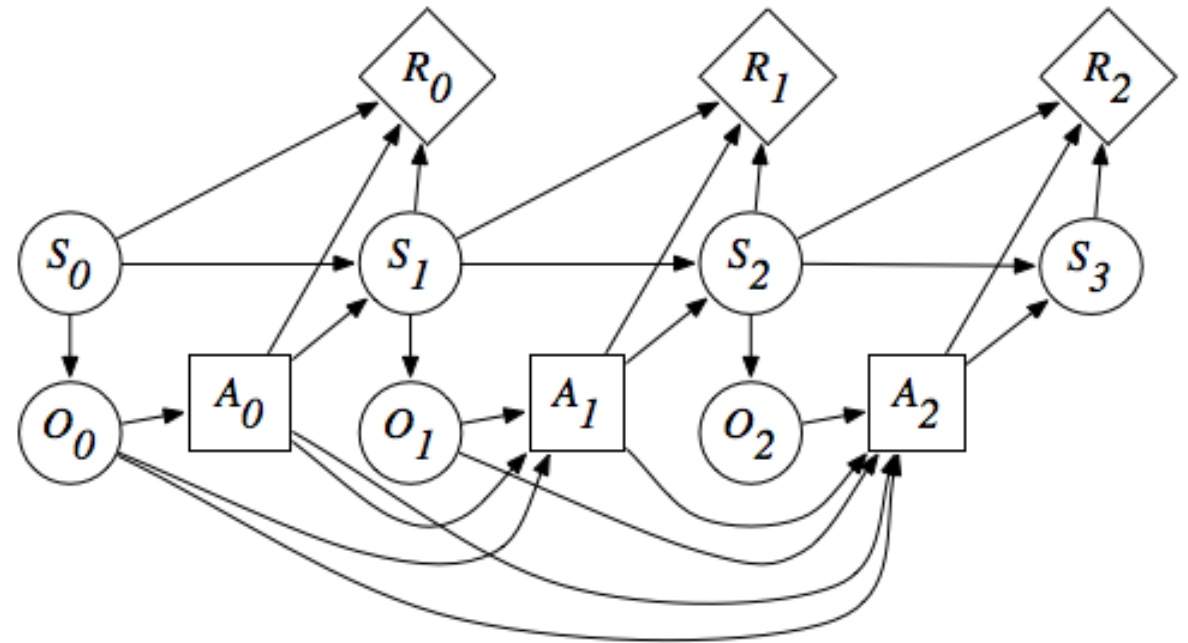
- Markov property:
 - The transition to s_t only depends on the immediately preceding state and action, s_{t-1} and a_{t-1} , not at all on earlier states and actions.
 - The state must include information about all aspects of the past agent–environment interaction that make a difference for the future

MDP vs. POMDP

Markov Decision Process

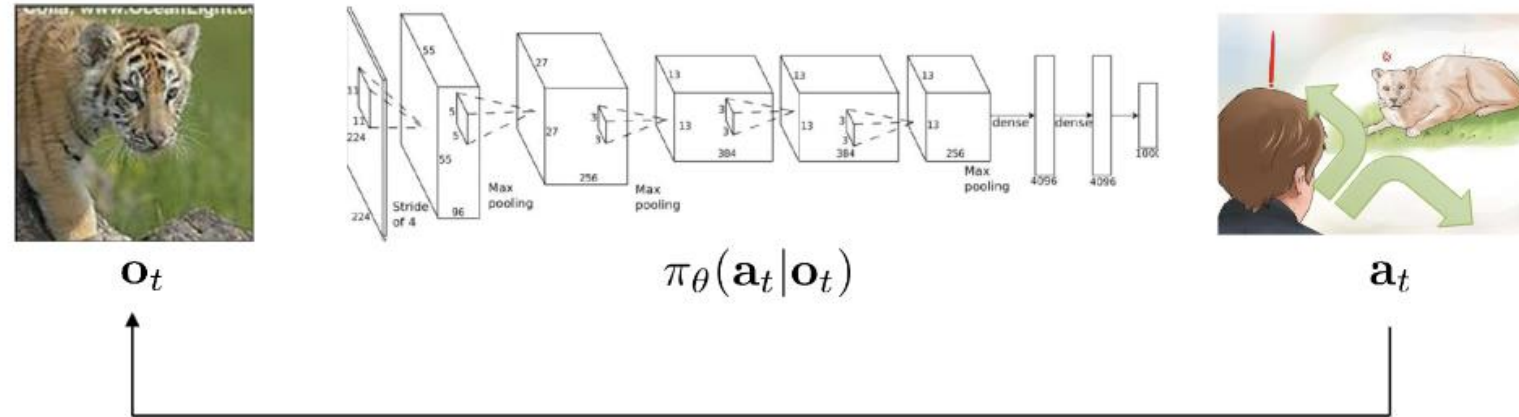


Partial Observable Markov Decision Process



State is a hidden variable. The decision is only conditioned on partial observations

(Partially Observable) Markov Decision Process



The observed information of the environment

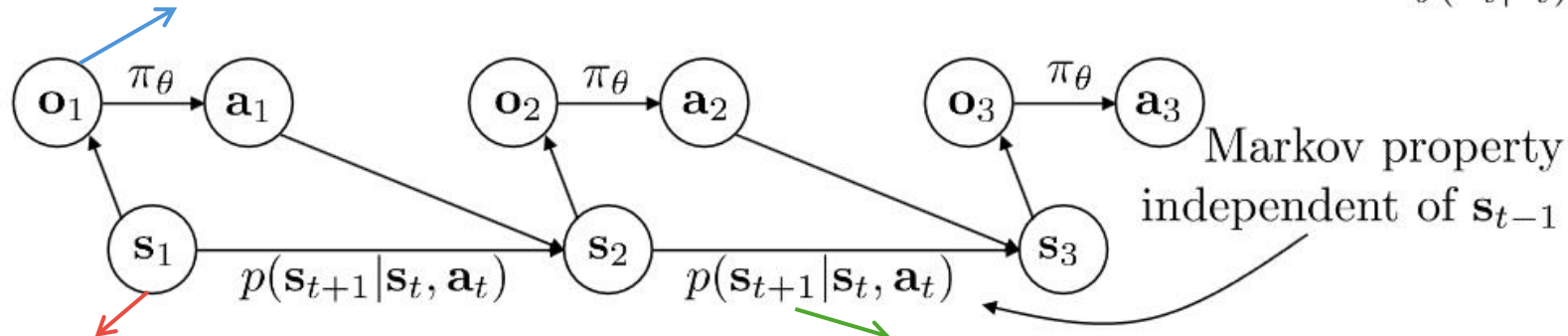
$\mathbf{s}_t$ – state

$\mathbf{o}_t$ – observation

$\mathbf{a}_t$ – action

$\pi_{\theta}(\mathbf{a}_t | \mathbf{o}_t)$ – policy

$\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ – policy (fully observed)



The true information of the environment

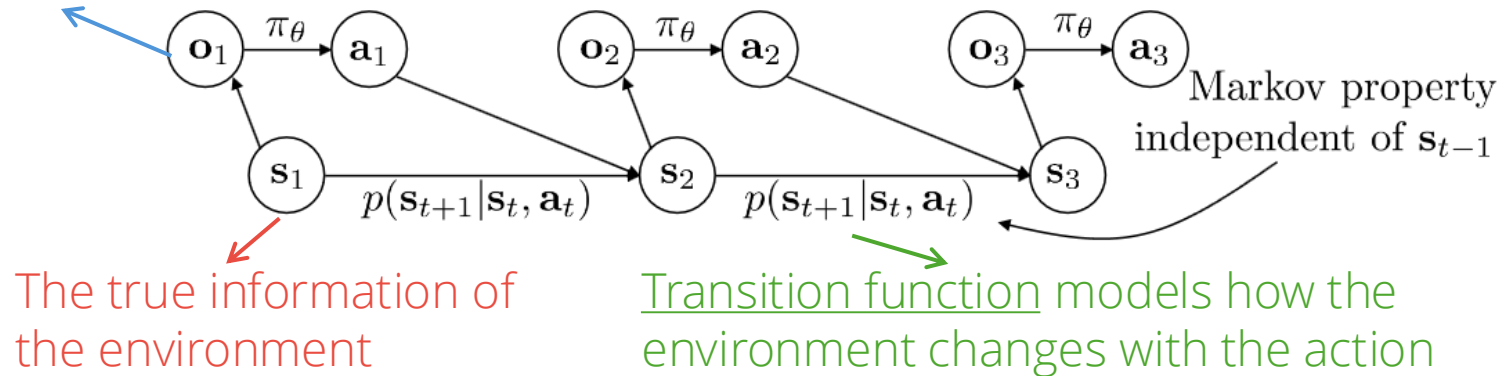
Transition function models how the environment changes with the action

Reward function signals if the goal is achieved:

- $r_t = r(\mathbf{s}_{t+1}, \mathbf{s}_t, \mathbf{a}_t)$
- $r_t = r(\mathbf{s}_t, \mathbf{a}_t)$

Markov Decision Process

The observed information of the environment



Reward function signals if the goal is achieved:

- $r_t = r(s_{t+1}, s_t, a_t)$
- $r_t = r(s_t, a_t)$

Most General Case:

Transition function is **unknown**

Reward function is **unknown**

Observed information is **incomplete** ($o_t \neq s_t$)

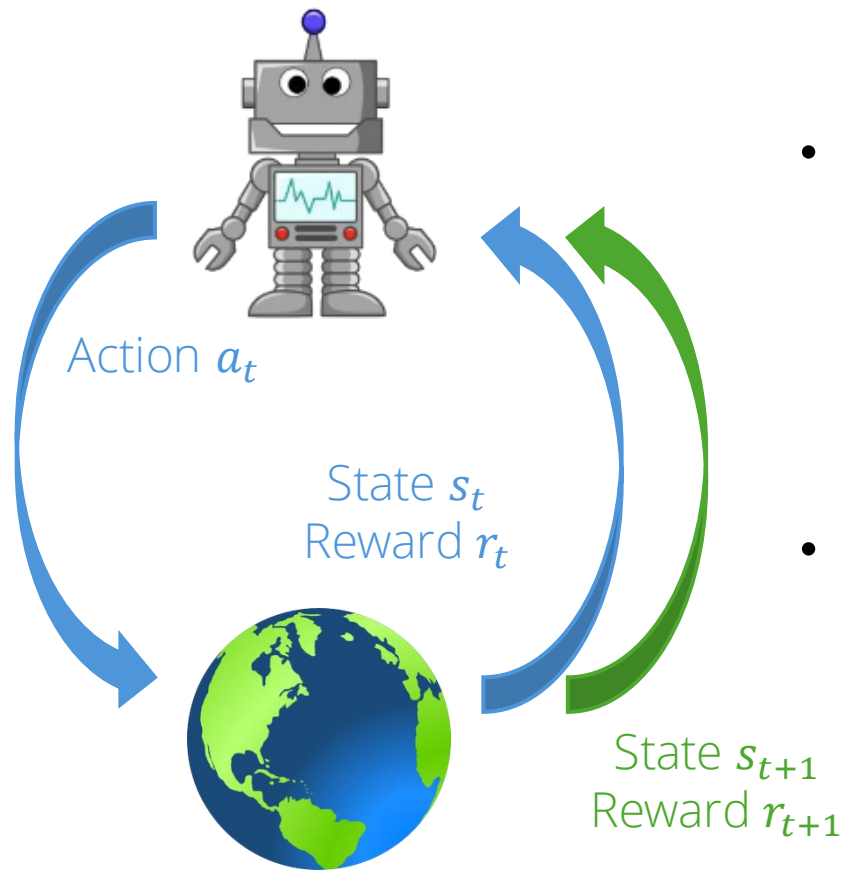
Most Specific Case:

Transition function is **known**

Reward function is **known**

Observed information is **complete** ($o_t = s_t$)

Reinforcement Learning: Learns a Policy that Maximizes the Total Reward of a sequence of actions



- A trajectory of interaction in the environment

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

- Maximize the expected value of the cumulative sum of reward

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

- Policy may be stochastic
- Environment may be stochastic
- Initial state may be randomly sampled
- $E_{\pi_{\theta}}[r(x)]$ is smooth in θ

Remember We Talked about Optimal Control

- Constrained Optimal Control Problem:

$$\min_{x,u} \sum_{k=0}^{N-1} c(x_k, u_k) + c_f(x_N)$$

$$s.t \ x_0 = \hat{x}_0$$

$$x_{k+1} = f(x_k, u_k), k = 0, \dots, N - 1$$

- Solution 1: Consider it as a mathematical optimization problem. We can solve it with existing optimization toolboxes (e.g. linear programming, quadratic programming, non-linear programming ...)
- Solution 2: Solve it with dynamic programming!

Remember We Talked about Optimal Control

- Constrained Optimal Control Problem:

$$\min_{x,u} \sum_{k=0}^{N-1} c(x_k, u_k) + c_f(x_N)$$

$$s.t. \ x_0 = \hat{x}_0$$

$$x_{k+1} = f(x_k, u_k), k = 0, \dots, N-1$$

The objective of RL:

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(s_t, a_t) \right]$$

In fact, they are very similar problems with different notations:

- control $u \leftrightarrow$ action a
 - state $x \leftrightarrow$ state s
 - cost $c(x, u) \leftrightarrow$ reward $r(s, a)$
 - minimize cost $\leftrightarrow$ maximize reward
- Solution 1: Consider it as a mathematical optimization problem. We can solve it with existing optimization toolboxes (e.g. linear programming, quadratic programming, non-linear programming ...)
 - Solution 2: Solve it with dynamic programming!

Content

- Formulation of reinforcement learning
- Multi-arm Bandit Problem
 - Exploration: Act randomly to explore information as much as possible
 - Exploitation: Act greedily to perform as best as possible
- Markovian Decision Making
 - Value estimation with Dynamic Programming
 - Policy Improvement
- Value estimation with Monte Carlo approach
- Value estimation with Temporal-Difference learning

Returns G_t in Episodic Tasks

- **Episode:** A sequence of interactions based on which the reward will be judged at the end
- **Episodic tasks:** interaction breaks naturally into episodes, e.g., plays of a game, trips through a maze
- **Returns G_t :** the cumulative sum of reward, or total reward, starting from time t
- Let **T** be the final time step:

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T$$

Returns G_t in Continuing Tasks

- **Continuing tasks:** interaction does not have natural episodes, but just goes on and on...just like real life
- **Discounted returns G_t :** the cumulative sum of reward, weighted by the discount rate γ
- Let T be the final time step, where $T \rightarrow \infty$:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- Discount rate γ decides the present value of future rewards.
 - If $\gamma = 0$, model is “myopic” concerning only the immediate reward
 - If $\gamma = 1$, model is “farsighted” ignoring the efficiency of achieving the goal

Value Functions are Expected Returns

- The state value function $v_{\pi}(s)$: the expected return starting from state s following policy π

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

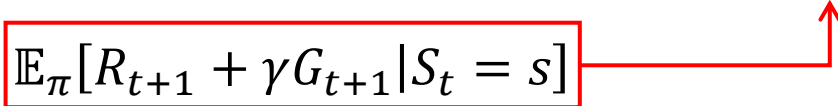
- The state value function is dependent on the deployed policy, since different actions lead to varied future states and total rewards.

The Recursive Relationships of Value Functions

- The state value function $v_\pi(s)$: the expected return starting from state s following policy π

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = R_{t+1} + \gamma G_{t+1}$



- The state value function is dependent on the deployed policy, since different actions lead to varied future states and total rewards.

The Recursive Relationships of Value Functions

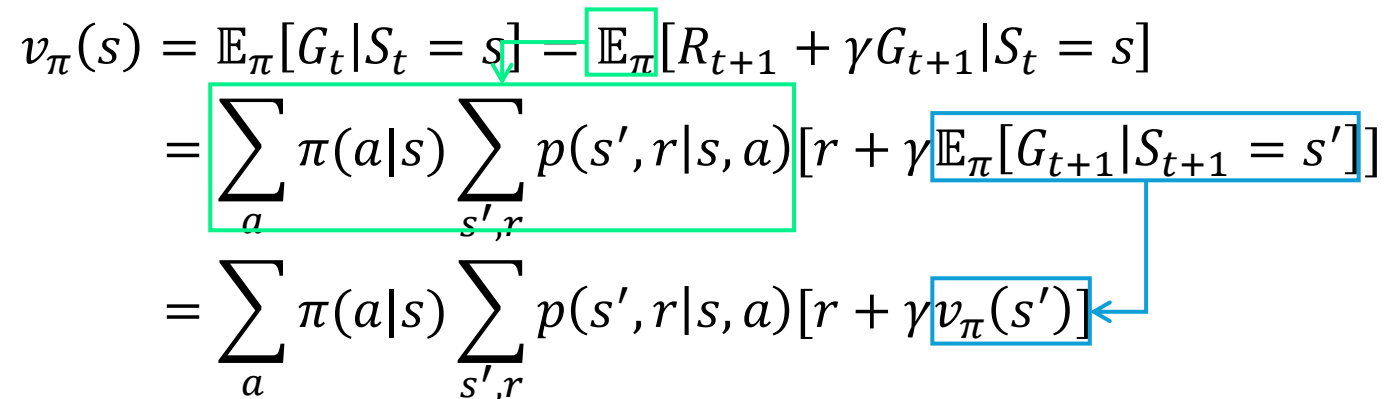
- The state value function $v_\pi(s)$: the expected return starting from state s following policy π

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s']] \end{aligned}$$

- The state value function is dependent on the deployed policy, since different actions lead to varied future states and total rewards.
- The state value function $v_\pi(s)$ estimates the “goodness” of state s when deploying policy π . Simply speaking, how easy it is to achieve the goal from state s .

The Recursive Relationships of Value Functions

- The state value function $v_\pi(s)$: the expected return starting from state s following policy π

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s']] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] \end{aligned}$$


- The state value function is dependent on the deployed policy, since different actions lead to varied future states and total rewards.

Value Functions are Expected Returns

- The action value function $q_\pi(s, a)$: the expected return starting from state s and action a following policy π

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')]$$

- The action-value function $q_\pi(s, a)$ estimates the “goodness” of action a at state s when deploying policy π . Simply speaking, how easy it is to achieve the goal with action a at state s .
- It's obvious to see

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] = \sum_a \pi(a|s) q_\pi(s, a)$$

Optimal Value Functions

- **Definition:** The optimal state-value function $v^*(s)$ is the maximum state-value function over all policies. In other words, the optimal value function specifies the best possible performance in the MDP.

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

- **Definition:** The optimal action-value function $q^*(s, a)$ is the maximum action-value function over all policies

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

Optimal Value Functions

- **Definition:** The optimal state-value function $v^*(s)$ is the maximum state-value function over all policies. In other words, the optimal value function specifies the best possible performance in the MDP.

$$v^*(s) = \max_{\pi} v_{\pi}(s)$$

- **Definition:** The optimal action-value function $q^*(s, a)$ is the maximum action-value function over all policies

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a)$$

- The optimal value function specifies the best possible performance in the MDP.
- An MDP is “solved” when we know the optimal value function

Value Functions

- Value functions measure the goodness of a particular state or state/action pair: how good is for the agent to be in a particular state or execute a particular action at a particular state, for a given policy.
- Optimal value functions measure the best possible goodness of states or state/action pairs under all possible policies.

	state values	action values
prediction	V_{π}	q_{π}
control	V_{*}	q_{*}

Why Value Functions are useful

Value functions capture the knowledge of the agent regarding how good is each state for the goal she is trying to achieve.

“...knowledge is represented as a large number of approximate value functions learned in parallel...”

Horde: A Scalable Real-time Architecture for Learning Knowledge from Unsupervised Sensorimotor Interaction, Sutton *et al.*

We Have the Optimal Policy if We Know $q^*(s, a)$

- An optimal policy can be found by maximizing over $q^*(s, a)$:

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \underset{a}{\operatorname{argmax}} q^*(s, a). \\ 0, & \text{otherwise.} \end{cases}$$

We Have the Optimal Policy if We Know $q^*(s, a)$

- An optimal policy can be found by maximising over $q^*(s, a)$:

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \underset{a}{\operatorname{argmax}} q^*(s, a). \\ 0, & \text{otherwise.} \end{cases}$$

- An optimal policy can be found by maximizing over $v^*(s)$ with one-step look ahead:

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \underset{a}{\operatorname{argmax}} (\sum_{s', r} p(s', r|s, a))(r + \gamma v_*(s')) \\ 0, & \text{otherwise.} \end{cases}$$

Bellman Optimality Equation for v^*

1. We have $v^*(s) = \max_a q_{\pi^*}(s, a)$

Bellman Optimality Equation for v^*

1. We have $v^*(s) = \max_a q_{\pi^*}(s, a)$

Why?

$$v_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) q_{\pi}(s, a)$$
$$\Rightarrow v^*(s) = \sum_{a \in \mathcal{A}} \pi^*(a|s) q_{\pi^*}(s, a) = \max_a q_{\pi^*}(s, a)$$

$$\pi^*(a|s) = \begin{cases} 1, & \text{if } a = \operatorname{argmax}_a q^*(s, a). \\ 0, & \text{otherwise.} \end{cases}$$

Bellman Optimality Equation for v^*

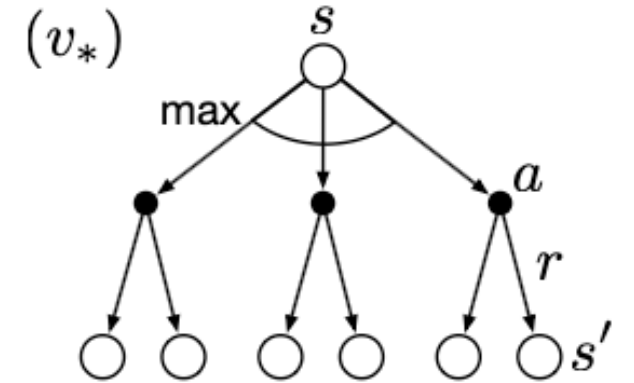
1. We have $v^*(s) = \max_a q_{\pi^*}(s, a)$
2. We can further write the equation:

$$\begin{aligned} v^*(s) &= \max_a q_{\pi^*}(s, a) \\ &= \max_a \mathbb{E}_{\pi^*}[G_t | S_t = s, A_t = a] \\ &= \max_a \mathbb{E}_{\pi^*}[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a] \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1}) | S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v^*(s')] \end{aligned}$$

Bellman Optimality Equation for v^*

1. We have $v^*(s) = \max_a q_{\pi^*}(s, a)$
2. We can further write the equation:

$$\begin{aligned} v^*(s) &= \max_a q_{\pi^*}(s, a) \\ &= \max_a \mathbb{E}_{\pi^*}[G_t | S_t = s, A_t = a] \\ &= \max_a \mathbb{E}_{\pi^*}[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a] \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1}) | S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma v^*(s')] \end{aligned}$$



This is again the recursive relationship!

How to Calculate Value Functions?

Iterative estimation: utilize the recursive relationships

Iterative Policy Evaluation

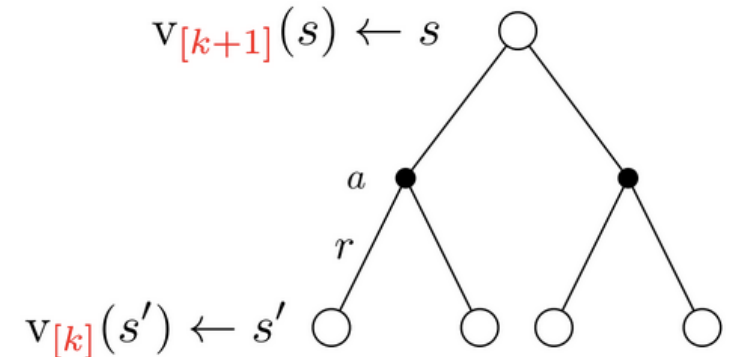
- The state value function $v_\pi(s)$:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_\pi(s')]$$

- We can utilize the recursive relationship to update $v_\pi(s)$

for $k = 1 \dots \infty$:

$$v_{k+1}(s) := \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_k(s')]$$



Iterative Policy Evaluation

- We can utilize the recursive relationship to update $v_{\pi}(s)$

for $k = 1 \dots \infty$:

$$v_{k+1}(s) := \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_k(s')]$$

- Dynamic programming solution: use a table to keep track of the value function

v_k

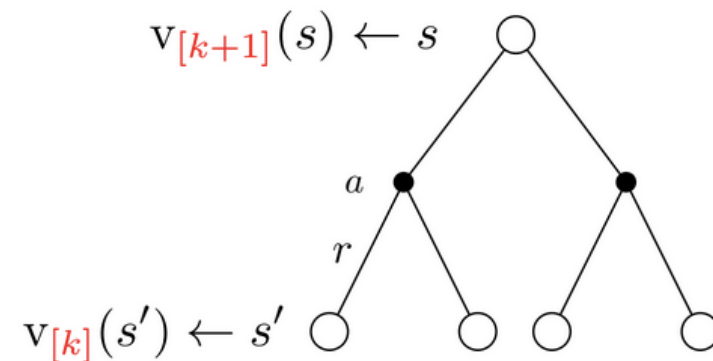
0.64	0.81	0.9	1
0.58	X	0.81	0.9
0.52	0	X	0.81
0	0	X	0.64

→

v_{k+1}

0.64	0.81	0.9	1
0.58	X	0.81	0.9
0.52	0.47	X	0.81
0.47	0	X	0.64

update



Estimate Value Function with Dynamic Programming

- Current policy: going up, down, left, right with even probability. The action that would take the agent off the grid would leave the state unchanged.
- Task: Travel from the **start** to the **goal** location
- Reward: reach **goal** = 1, otherwise 0

Reward map

v_0

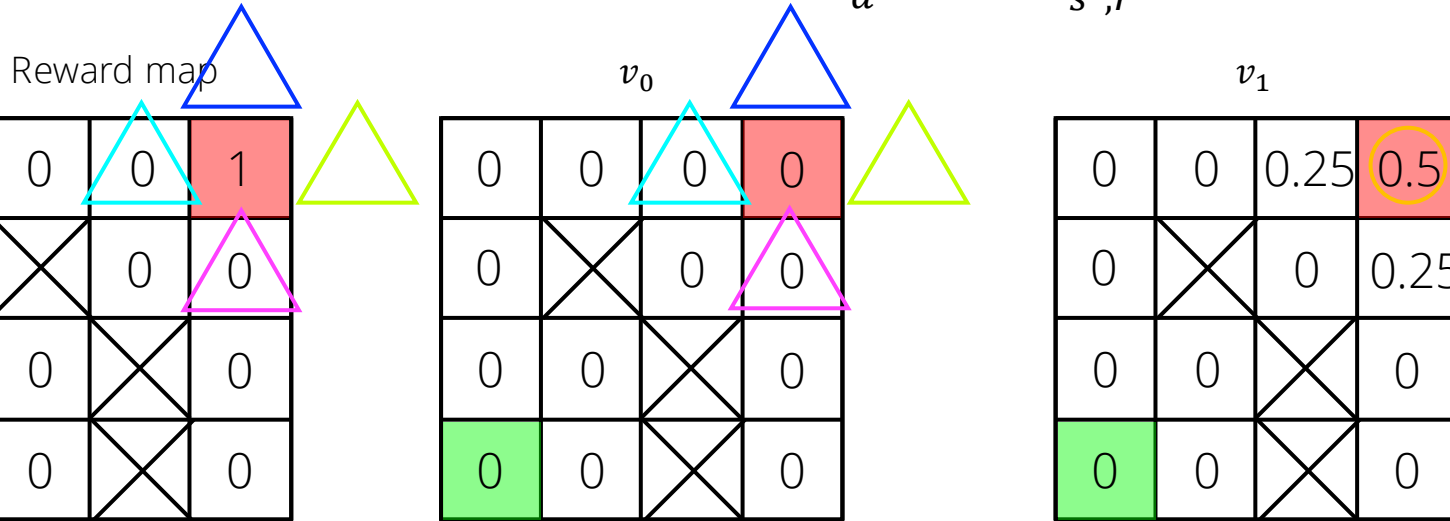
v_1

0	0	0	1	0	0	0	0	0	0	0.25	0.5
0	X	0	0	0	X	0	0	0	X	0	0.25
0	0	X	0	0	0	X	0	0	0	X	0
0	0	X	0	0	0	X	0	0	0	X	0

Estimate Value Function with Dynamic Programming

- Backward dynamic programming, let $\gamma = 1.0$ (no discount)

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_k(s')]$$



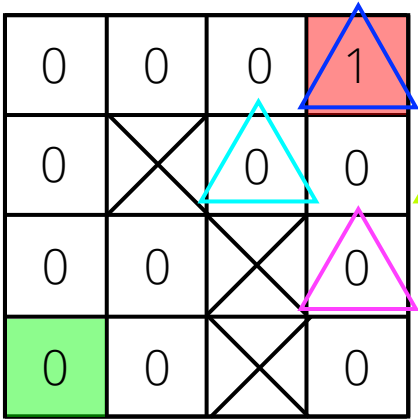
$$0.5 = \frac{1}{4} \times (1 + 1 \times 0) + \frac{1}{4} \times (1 + 1 \times 0) + \frac{1}{4} \times (0 + 1 \times 0) + \frac{1}{4} \times (0 + 1 \times 0)$$

Estimate Value Function with Dynamic Programming

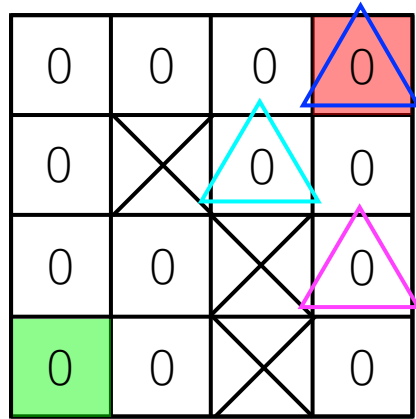
- Backward dynamic programming, let $\gamma = 1.0$ (no discount)

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_k(s')]$$

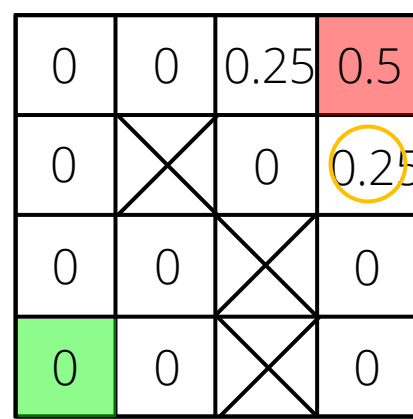
Reward map



v_0



v_1



$$0.25 = \frac{1}{4} \times (1 + 1 \times 0) + \frac{1}{4} \times (0 + 1 \times 0) + \frac{1}{4} \times (0 + 1 \times 0) + \frac{1}{4} \times (0 + 1 \times 0)$$

Estimate Value Function with Dynamic Programming

- Backward dynamic programming, let $\gamma = 1.0$ (no discount)

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_k(s')]$$

Reward map

0	0	0	1
0	X	0	0
0	0	X	0
0	0	X	0

v_1

0	0	0.25	0.5
0	X	0	0.25
0	0	X	0
0	0	X	0

v_2

0	0.06	0.44	0.88
0	X	0.13	0.44
0	0	X	0.06
0	0	X	0

$$0.44 = \frac{1}{4} \times (1 + 1 \times 0.5) + \frac{1}{4} \times (0 + 1 \times 0.25) + \frac{1}{4} \times (0 + 1 \times 0) + \frac{1}{4} \times (0 + 1 \times 0)$$

Iterative Policy Evaluation

$$v_{k+1}(s) := \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_k(s')]$$

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Content

- Formulation of reinforcement learning
- Multi-arm Bandit Problem
 - Exploration: Act randomly to explore information as much as possible
 - Exploitation: Act greedily to perform as best as possible
- Markovian Decision Making
 - Value estimation with Dynamic Programming
 - Policy Improvement
- Value estimation with Monte Carlo approach
- Value estimation with Temporal-Difference learning

Can We Improve the Current Policy?

- Let's say we obtain the value function $v_\pi(s)$ based on policy π using dynamic programming, How can we improve the policy?
- Switch to a greedy policy!

$$\pi'(a|s) = \begin{cases} 1, & \text{if } a = \underset{a}{\operatorname{argmax}}(\sum_{s',r} p(s',r|s,a)(r + \gamma v_\pi(s'))) \\ 0, & \text{otherwise.} \end{cases}$$

- Why greedy policy π' is better than the original policy π at state s ?
Since a greedy policy is deterministic: $\pi'(s) = \underset{a}{\operatorname{argmax}}(\sum_{s',r} p(s',r|s,a)(r + \gamma v_\pi(s')))$

$$q_\pi(s|\pi'(s)) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma v_\pi(s')]$$

The value of selecting action $\pi'(s)$ is higher than following policy π at state s (here we still follow policy π at other states) $\geq \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_\pi(s')] = v_\pi(s)$

Check Appendix 5 for the proof!

Policy Evaluation and Policy Improvement

- Policy Evaluation: update the (state) value function following the current policy π

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_{\pi}(s')]$$

- Policy Improvement: improve the current policy π by acting greedily

$$\pi'(a|s) = \begin{cases} 1, & \text{if } a = \underset{a}{\operatorname{argmax}}(\sum_{s',r} p(s',r|s,a)(r + \gamma v_{\pi}(s'))) \\ 0, & \text{otherwise.} \end{cases}$$

- What if the new greedy policy π' is no better than the original policy π ($v_{\pi'} = v_{\pi}$)?

$$\begin{aligned} v_{\pi'}(s) &= \sum_a \pi'(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_{\pi'}(s')] \\ &= \max_a \left(\sum_{s',r} p(s',r|s,a)(r + \gamma v_{\pi'}(s')) \right) \end{aligned}$$

We have an optimal policy!

Can We Approach Optimality by Alternating Policy Evaluation and Improvement?

$$\pi_0 \xrightarrow{\boxed{\text{E}}} v_{\pi_0} \xrightarrow{\boxed{\text{I}}} \pi_1 \xrightarrow{\boxed{\text{E}}} v_{\pi_1} \xrightarrow{\boxed{\text{I}}} \pi_2 \xrightarrow{\boxed{\text{E}}} \dots \xrightarrow{\boxed{\text{I}}} \pi_* \xrightarrow{\boxed{\text{E}}} v_*,$$

- Policy Evaluation: update the (state) value function following the current policy π
- Policy Improvement: improve the current policy π by acting greedily

Policy Iteration

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*,$$

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

$policy_stable \leftarrow true$

For each $s \in \mathcal{S}$:

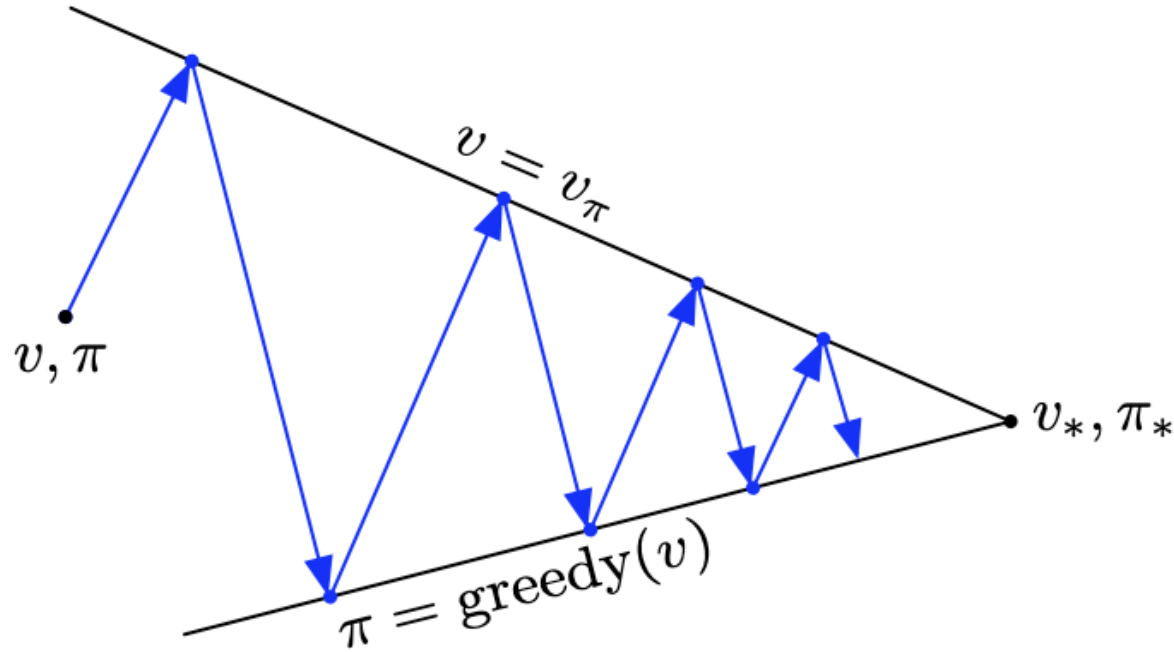
$old_action \leftarrow \pi(s)$

$\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

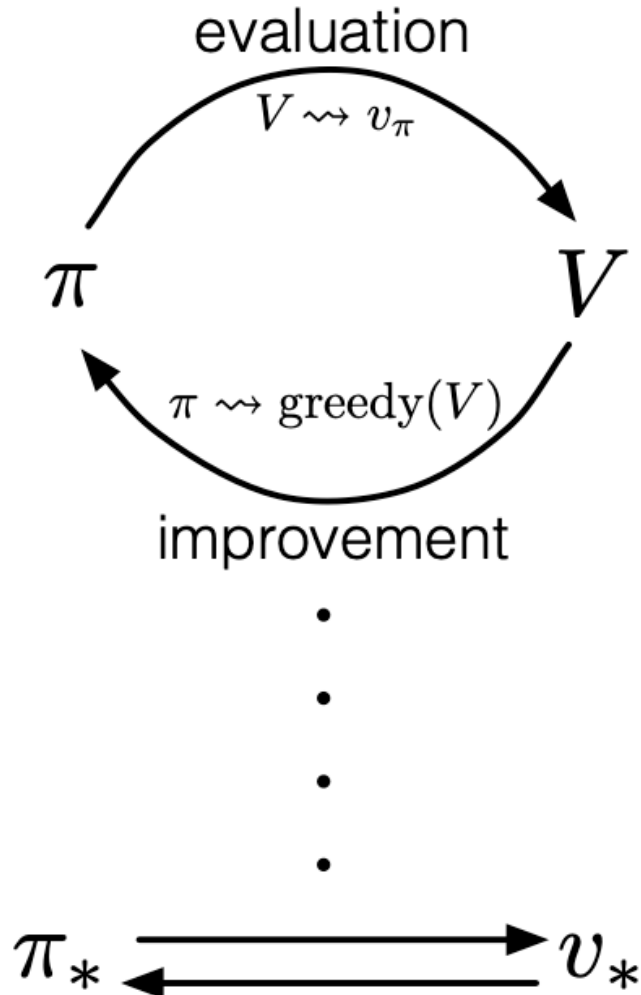
If $old_action \neq \pi(s)$, then $policy_stable \leftarrow false$

If $policy_stable$, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Generalized Policy Iteration



- **Generalized Policy Iteration:** general idea of letting policy-evaluation and policy-improvement processes interact, independent of the granularity and other details of the two processes



Value Iteration

Things still work out even if we are lazy and partially complete policy iteration steps

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$ 
| Loop for each  $s \in \mathcal{S}$ :
|    $v \leftarrow V(s)$ 
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$ 
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

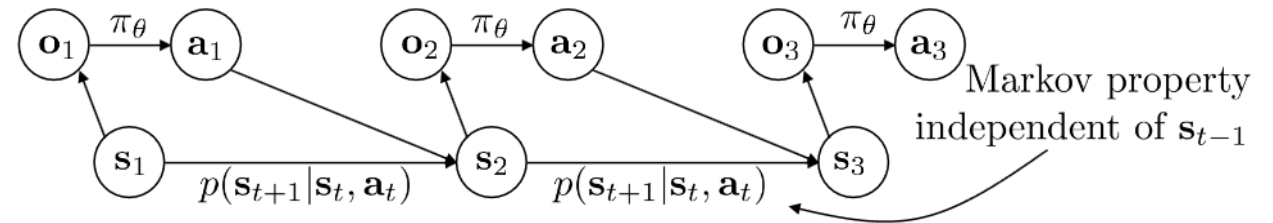
$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$

Check Appendix 6 for the proof!

So far, we have several assumptions

We can solve MDP if we know:

1. The transition function (dynamics) of the environment
2. The reward function
3. The Markov property holds
4. We have enough computational resource



Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

$\Delta \leftarrow 0$

 Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

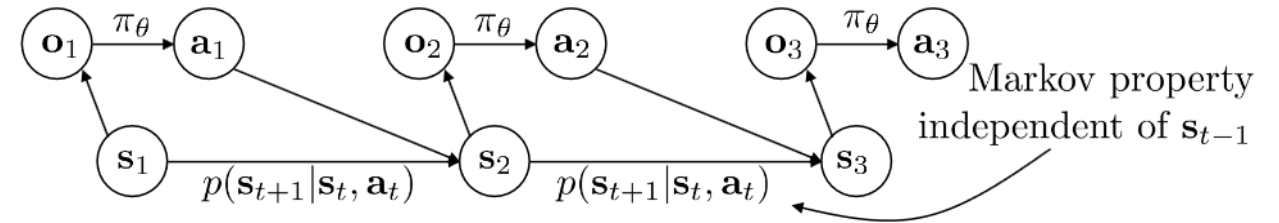
Output a deterministic policy, $\pi \approx \pi_*$, such that

$\pi(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

Next, we'll go beyond these assumptions

We can solve MDP if we know:

1. The transition function (dynamics) of the environment
2. The reward function
3. The Markov property holds
4. We have enough computational resource



Monte Carlo Methods!

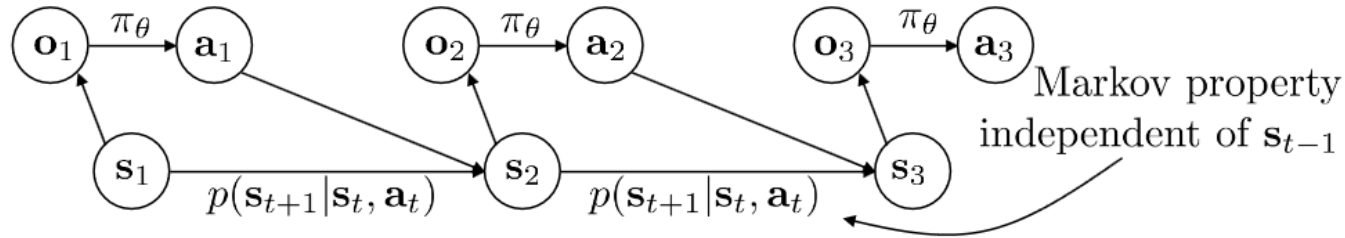
Temporal-Difference Learning!

Content

- Formulation of reinforcement learning
- Multi-arm Bandit Problem
 - Exploration: Act randomly to explore information as much as possible
 - Exploitation: Act greedily to perform as best as possible
- Markovian Decision Making
 - Value estimation with Dynamic Programming
 - Policy Improvement
- Value estimation with Monte Carlo approach
- Value estimation with Temporal-Difference learning

What if we don't know the transition function and reward function?

Markov Decision Processes:



Assumptions:



Value Functions

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} \cancel{p(s', r|s, a)} [r(s, a) + \gamma v_\pi(s')]$$

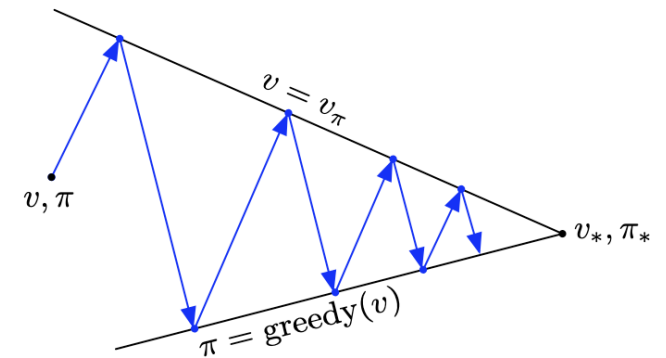
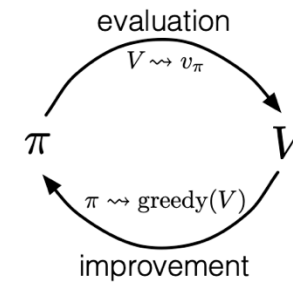
$$q_\pi(s, a) = \sum_{s', r} \cancel{p(s', r|s, a)} [r(s, a) + \gamma v_\pi(s')]$$

Bellman Optimality Equation

$$\begin{aligned} v_*(s) &= \max_a q_{\pi_*}(s, a) \\ &= \max_a \sum_{s', r} \cancel{p(s', r|s, a)} [r(s, a) + \gamma v_*(s')] \end{aligned}$$

$$q_*(s, a) = \sum_{s', r} \cancel{p(s', r|s, a)} [r(s, a) + \gamma \max_{a'} q_*(s', a')]$$

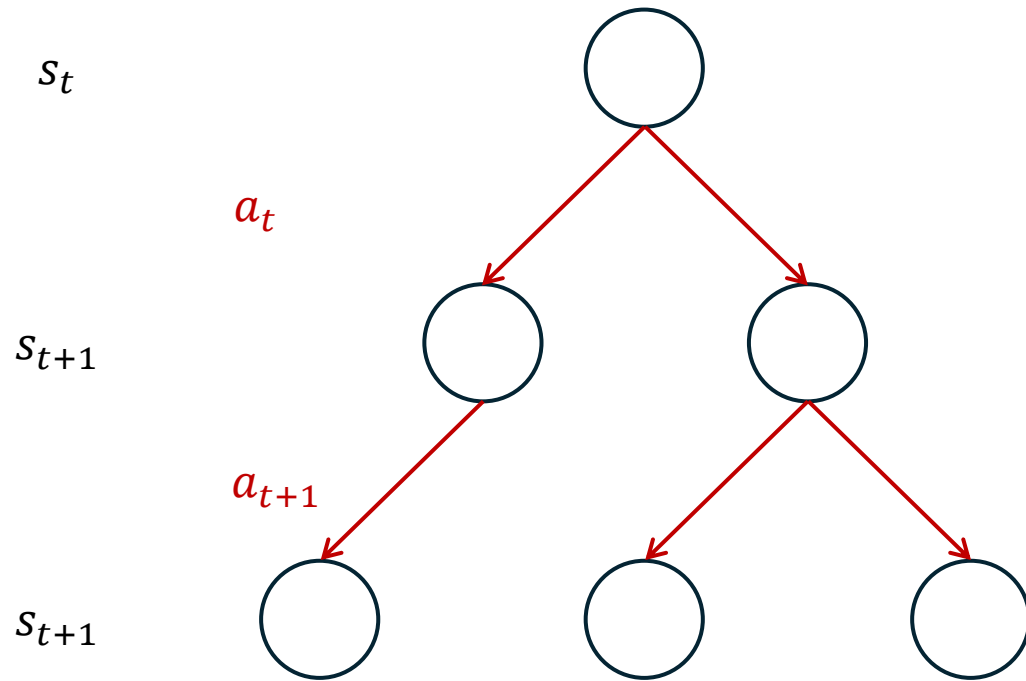
Generalized Policy Iteration



$$\pi_* \longleftrightarrow v_*$$

Monte Carlo (MC) Methods: Learning from experience

Dynamic Programming Method

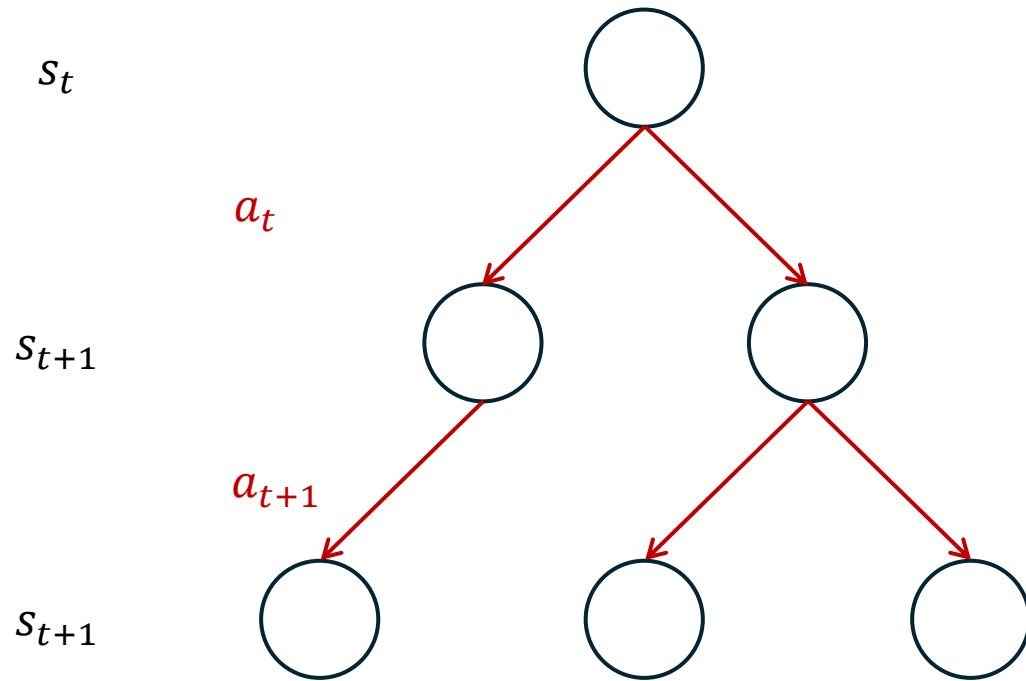


Simulated interaction: we know s_{t+1}
as we know $p(s_{t+1}|s_t, a_t)$

- No (need for) exploration
- No (need for) interaction

Monte Carlo (MC) Methods: Learning from experience

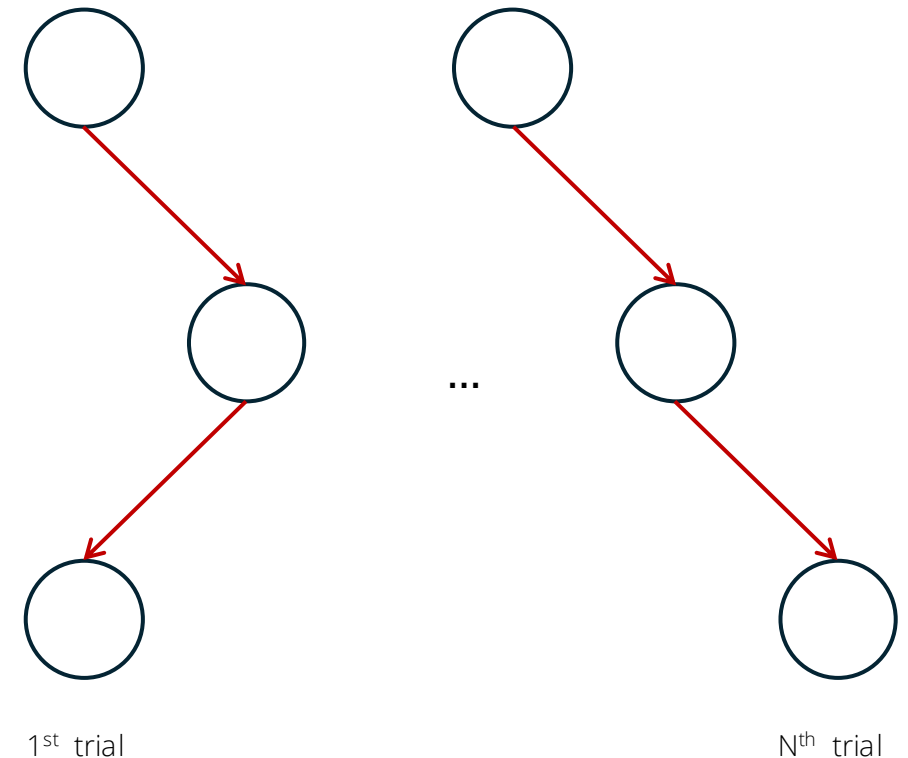
Dynamic Programming Method



Simulated interaction: we know s_{t+1}
as we know $p(s_{t+1}|s_t, a_t)$

- No (need for) exploration
- No (need for) interaction

Monte Carlo Method



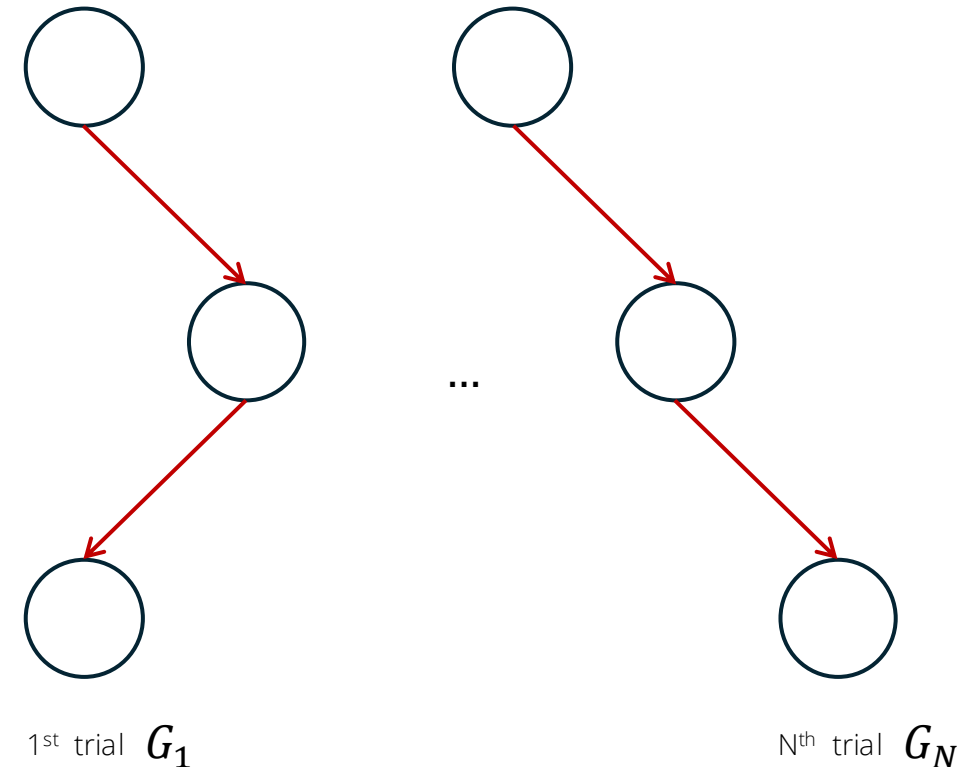
Actual experience: we don't know s_{t+1} .
unless we visit it

- Need exploration and interaction

Monte Carlo (MC) Methods: Learning from experience

- MC is model-free: no knowledge of MDP transitions / rewards
- MC learns from complete episodes: no bootstrapping (the estimates for each state is independent)
- What is “*bootstrapping*”?
 - Update value estimates on the basis of other estimates

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_{\pi}(s')]$$



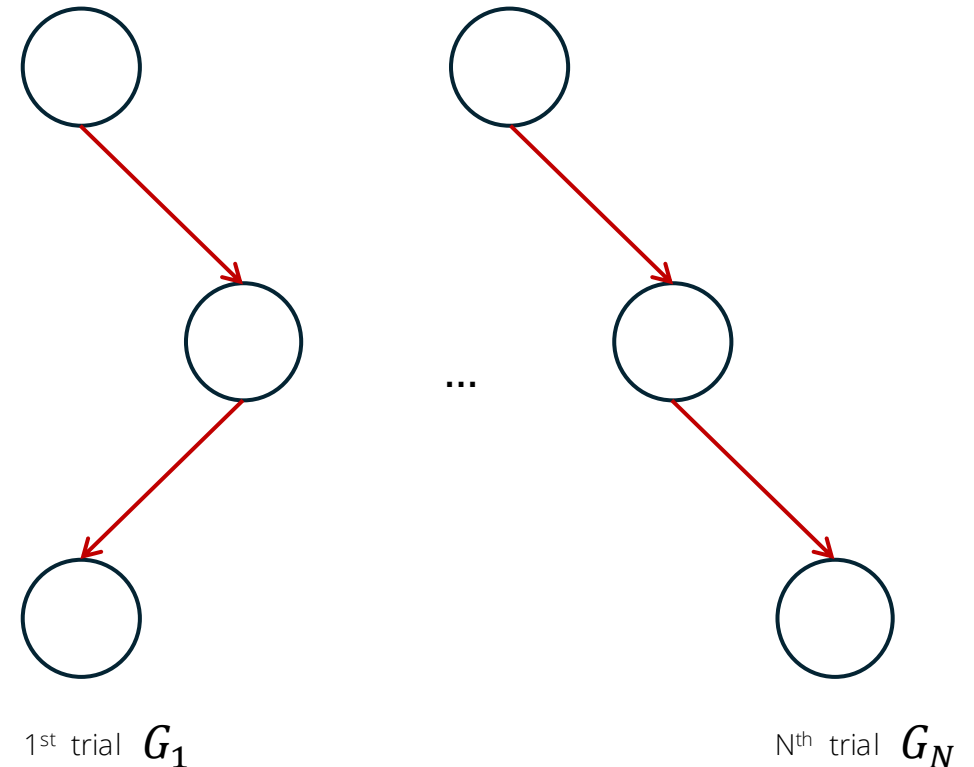
Monte Carlo (MC) Methods: Learning from experience

- How to estimate values?
 - Sample-average return:

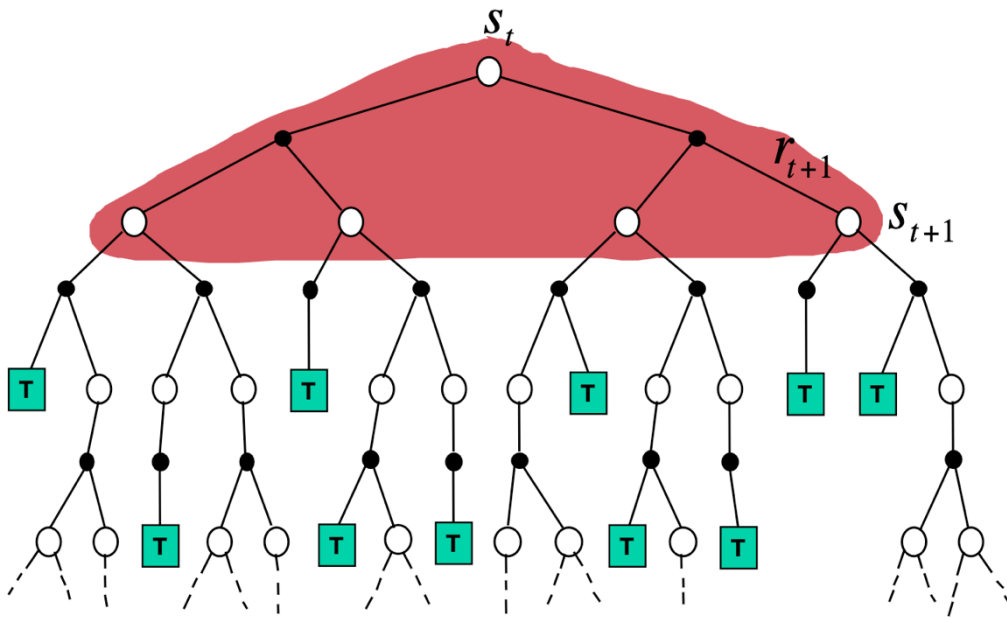
$$V_N(s) = \frac{G_1 + \dots + G_N}{N}$$

average observed returns from state s

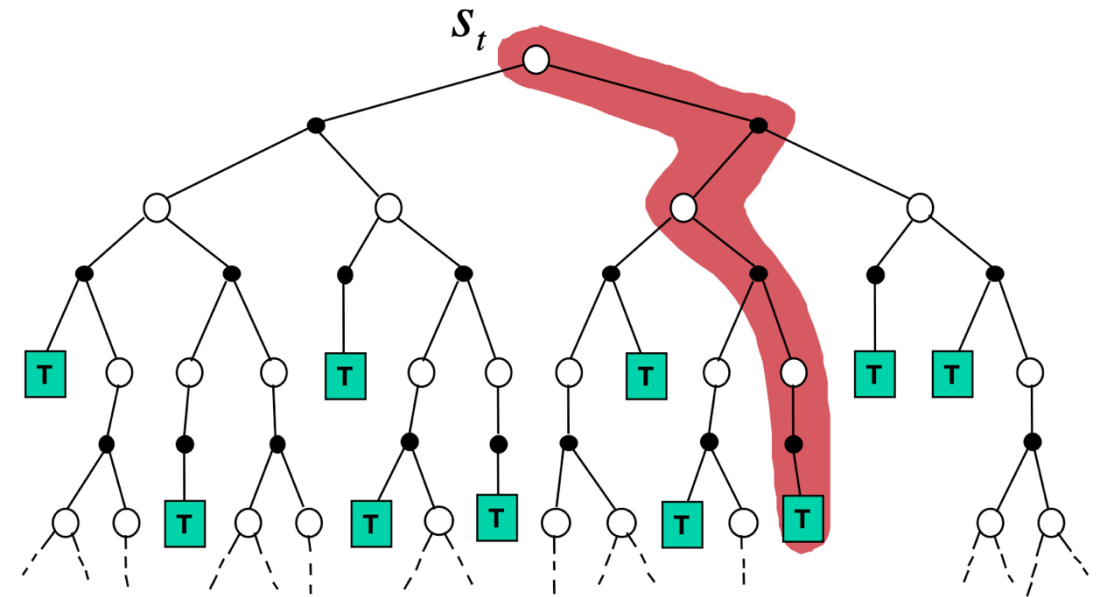
- An incremental implementation (check Appendix 1)



Value Estimation via DP vs. MC



Dynamic Programming

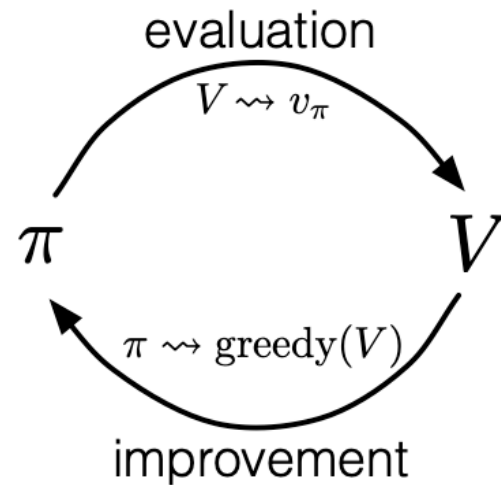


Monte Carlo

How to Obtain Optimal Policies with Monte Carlo methods?

The same idea as generalized policy iteration: alternates optimization of policy evaluation and policy improvement

$$v(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v(s')]$$



But we don't know $p(s', r | s, a)$!

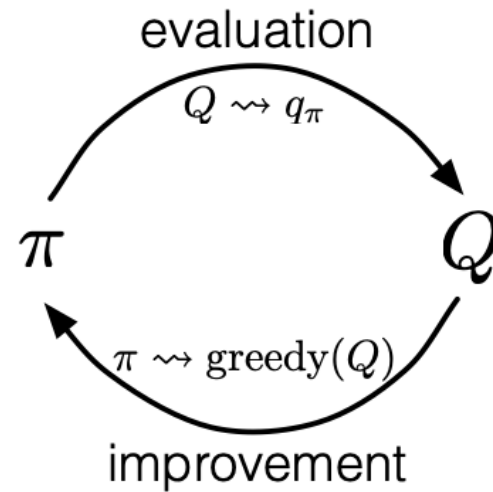
$$\pi(s) \leftarrow \underset{a}{\operatorname{argmax}} \sum_{s',r} \cancel{p(s',r | s, a)} [r + \gamma v_\pi(s')]$$

Convergence of Monte Carlo Control

Use state-action value $q_\pi(s, a)$, and we don't need to know $p(s', r|s, a)$!

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*,$$

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg\max_a q_{\pi_k}(s, a)) \\ &= \max_a q_{\pi_k}(s, a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \\ &\geq v_{\pi_k}(s). \end{aligned}$$



- MC methods converge, if:
 - We have infinite number of episodes (so the value estimate converges to the true value)
 - We visit every state-action pairs (so the value estimate will be the same as the true value)

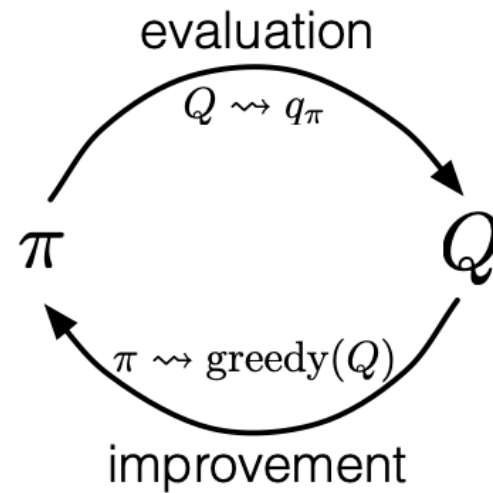
$$\pi(s) \leftarrow \arg\max_a q_\pi(s, a)$$

Convergence of Monte Carlo Control

Use state-action value $q_\pi(s, a)$, and we don't need to know $p(s', r|s, a)$!

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*,$$

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg\max_a q_{\pi_k}(s, a)) \\ &= \max_a q_{\pi_k}(s, a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \\ &\geq v_{\pi_k}(s). \end{aligned}$$



$$\pi(s) \leftarrow \arg\max_a q_\pi(s, a)$$

- MC methods converge, if:
 - We have infinite number of episodes (so the value estimate converges to the true value)
 - We visit every state-action pairs (so the value estimate will be the same as the true value)
- In other words, we need to **explore**!
 - If the policy always takes greedy action, we can never **explore** unseen state-action pairs

The Exploration-Exploitation Dilemma

- **Exploitation:** maximize the current highest reward: $\pi(s) = \underset{a}{\operatorname{argmax}} q_{\pi}(s, a)$
- **Exploration:** maximize the information about the environment

The Exploration-Exploitation Dilemma

- **Exploitation:** maximize the current highest reward: $\pi(s) = \underset{a}{\operatorname{argmax}} q_{\pi}(s, a)$
- **Exploration:** maximize the information about the environment
- **Solutions:**
 - exploring starts: Every state-action pair has a non-zero probability of being the starting pair
 - ϵ -soft policies: Most of the time choose the action with maximal estimated action values, but with probability ϵ select a random action
 - off-policy: use different policies for collecting experience and evaluating

Check Appendix 7 for more details!

Quick Summary: DP vs. MC

- Dynamic Programming (DP) methods are efficient, which **bootstrap** value functions from existing estimates.

$$V(S_t) \leftarrow \sum_{A_t} \pi(A_t|S_t) \sum_{S_{t+1}, R_{t+1}} p(S_{t+1}, R_{t+1}|S_t, A_t) [R_{t+1} + \gamma V(S_{t+1})]$$


- Monte Carlo (MC) methods: must wait until **the end of the episode** to learn value functions (only when the return is known)

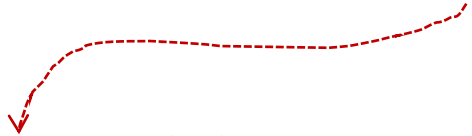
$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

Content

- Formulation of reinforcement learning
- Multi-arm Bandit Problem
 - Exploration: Act randomly to explore information as much as possible
 - Exploitation: Act greedily to perform as best as possible
- Markovian Decision Making
 - Value estimation with Dynamic Programming
 - Policy Improvement
- Value estimation with Monte Carlo approach
- Value estimation with Temporal-Difference learning

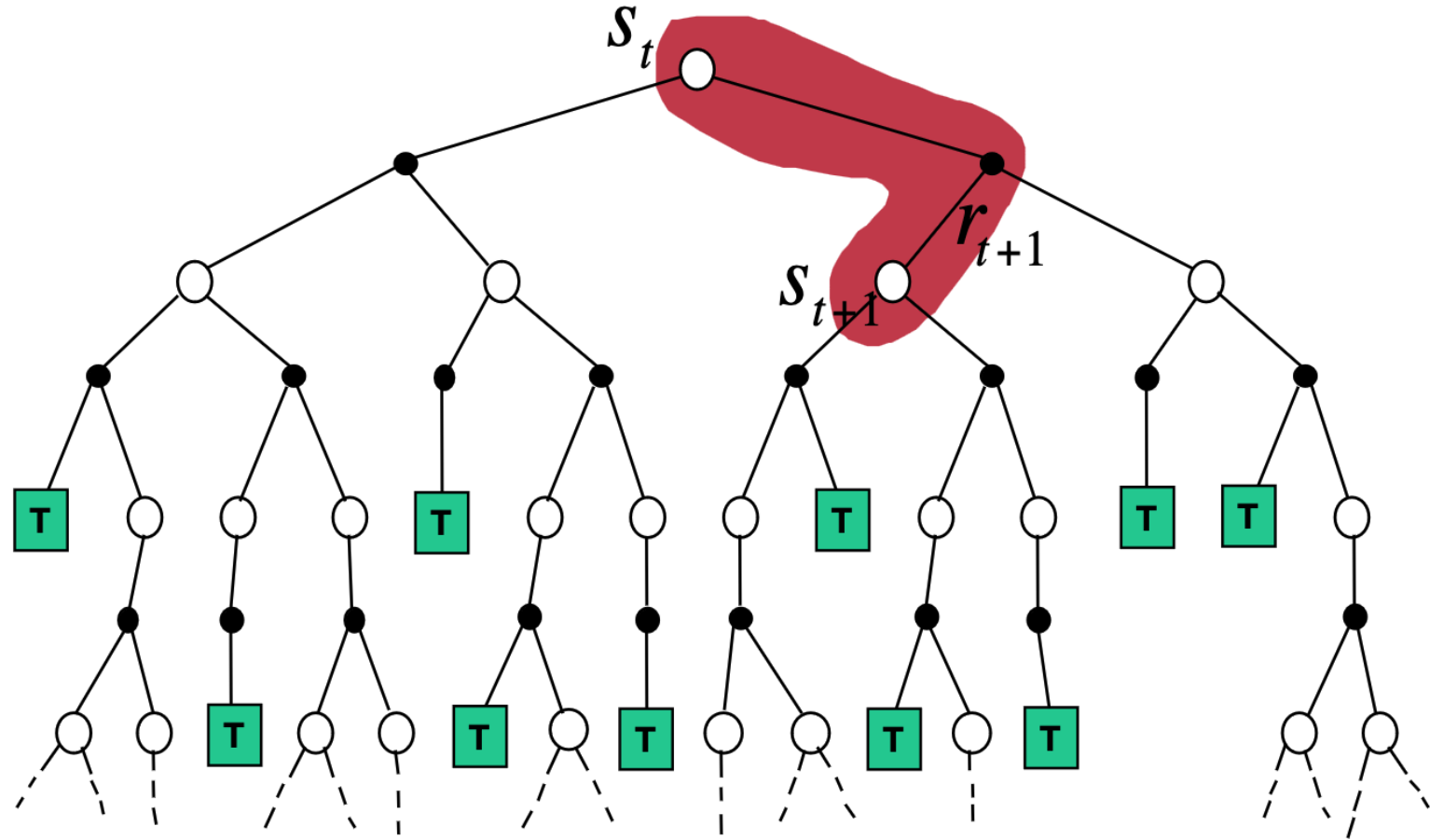
Temporal-Difference Learning

- Temporal-Difference (TD) methods: combine Monte Carlo methods with Dynamic Programming methods that wait only until the **next time step** and **bootstrap** value functions from existing estimates

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$


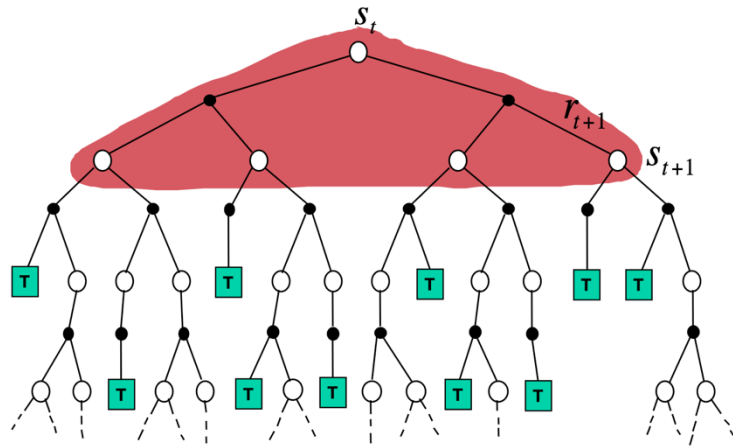
- TD target: $R_{t+1} + \gamma V(S_{t+1})$
- TD error: $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$

Value Estimation with Temporal-Difference Learning



Backup diagram: DP vs. MC vs. TD

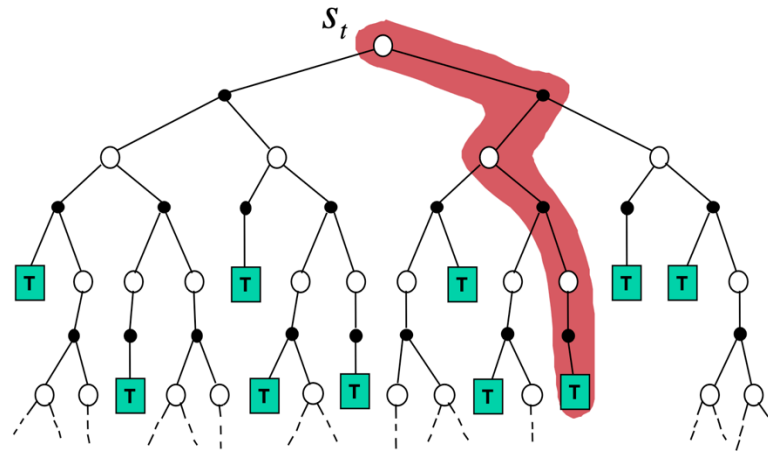
Dynamic Programming Backup



$$V(S_t) \leftarrow \sum_{A_t} \pi(A_t|S_t) \sum_{S_{t+1}, R_{t+1}} p(S_{t+1}, R_{t+1}|S_t, A_t) [R_{t+1} + \gamma V(S_{t+1})]$$

Expected updates: based on the summation of all successors

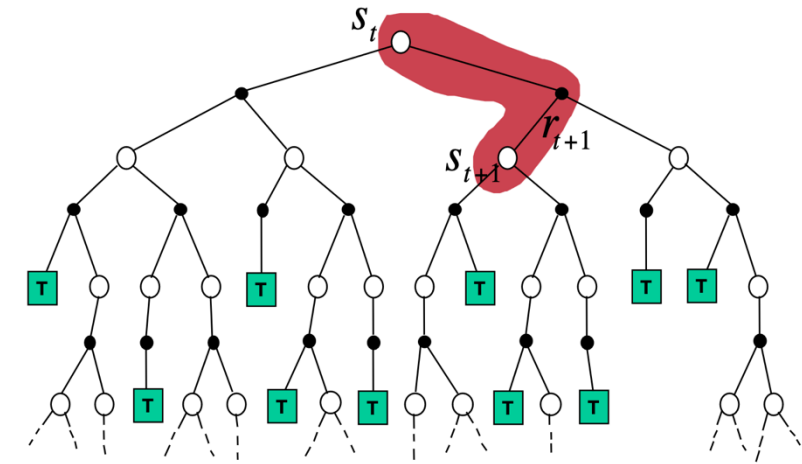
MC Backup



$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

Sample updates: based on a single sample successor

TD Backup

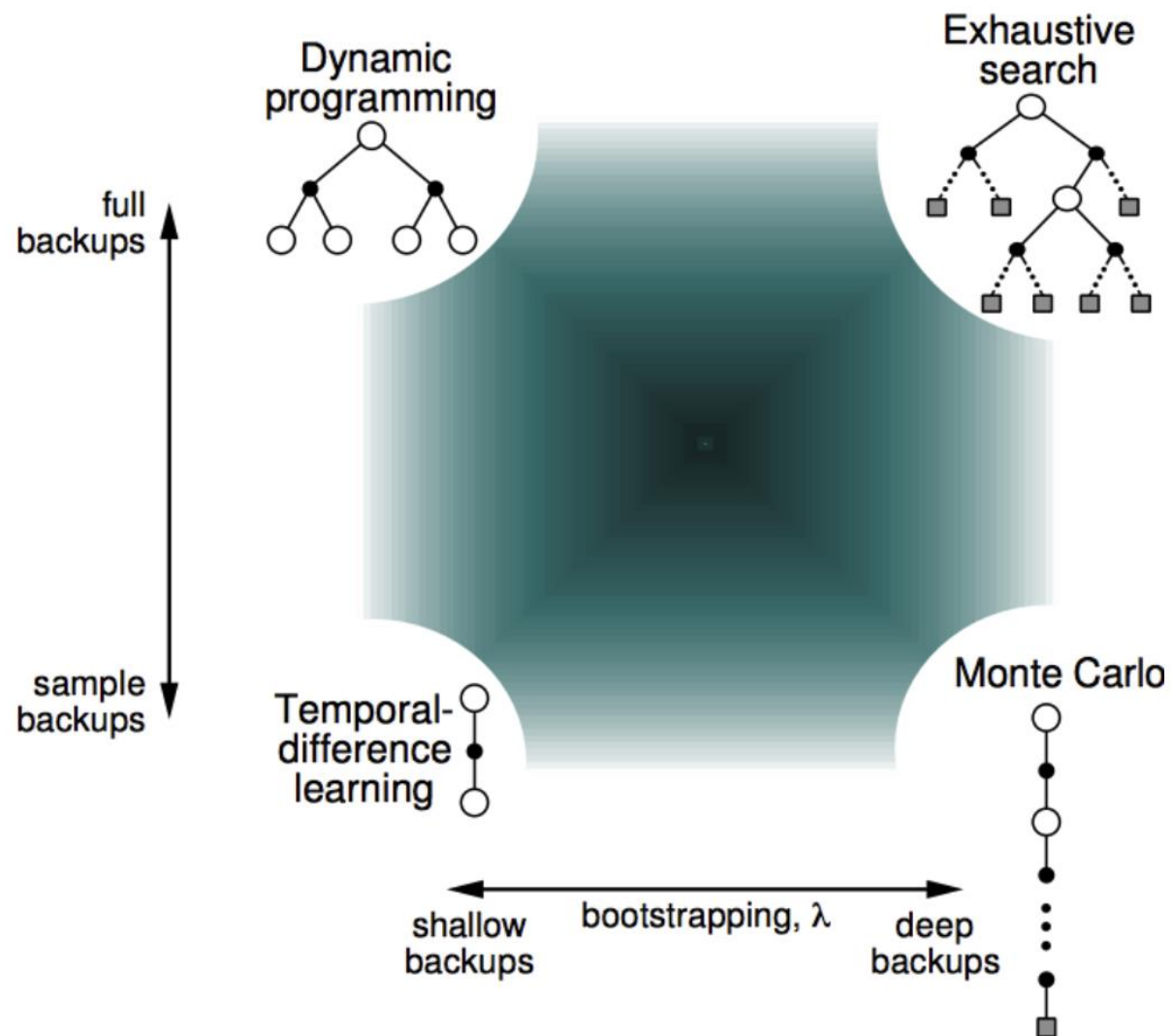


$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

DP vs. MC vs. TD

	Bootstrap	Sample
Dynamic Programming	✓	×
Monte Carlo	×	✓
Temporal Difference	✓	✓

DP vs. MC vs. TD



Nice Properties of TD

- TD can learn before knowing the final outcome
 - TD can learn online after every step
 - MC must wait until end of episode before return is known
- TD can learn without the final outcome
 - TD can learn from incomplete sequences
 - MC can only learn from complete sequences
 - TD works in continuing (non-terminating) environments
 - MC only works for episodic (terminating) environments
- Both TD and MC converge (under certain conditions). Which converge better / faster?

Tabular TD(0) Prediction

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated

Algorithm parameter: step size $\alpha \in (0, 1]$

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

$A \leftarrow$ action given by π for S

 Take action A , observe R, S'

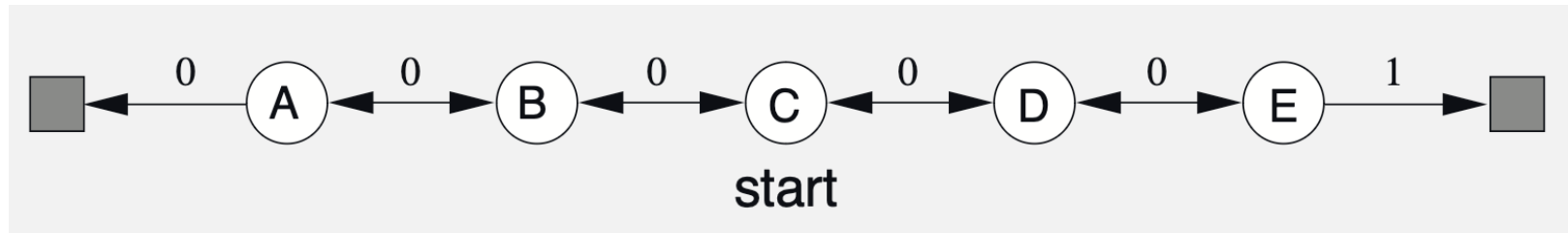
$V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

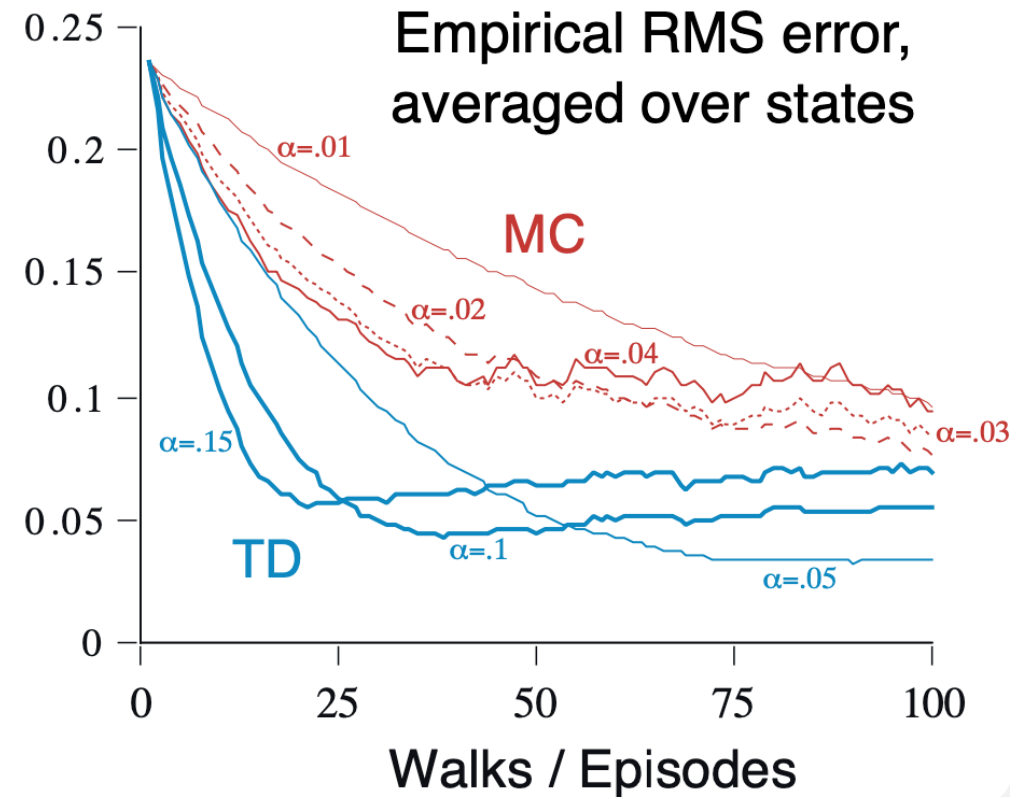
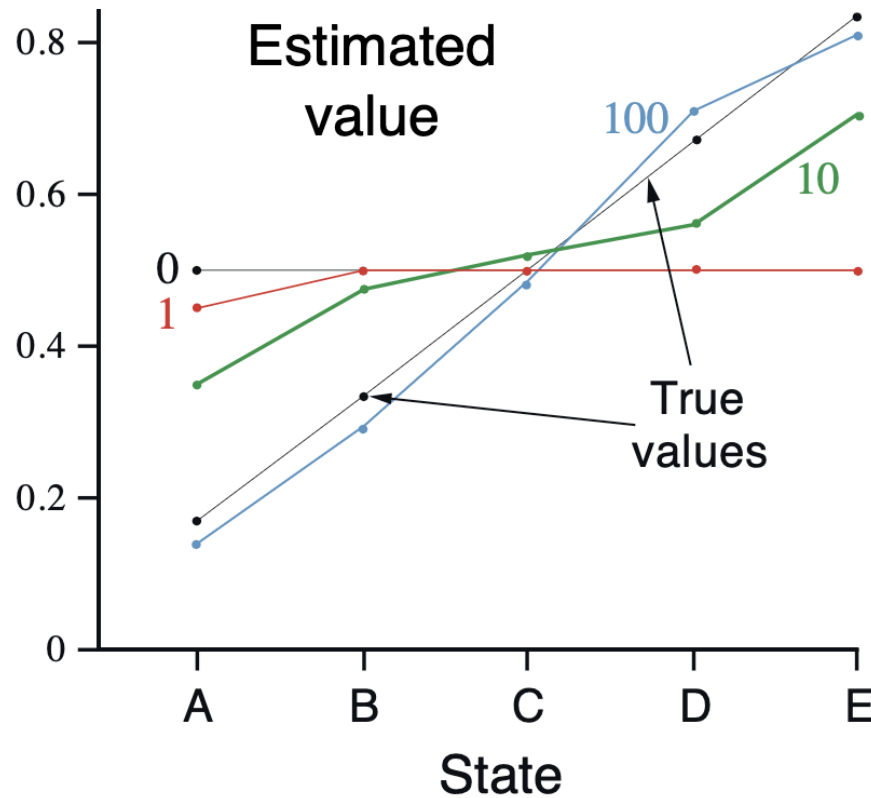
 until S is terminal

Random Walk Example

- Assume we have a Markov Reward Process—a MDP without actions.
- Always start from the center, moving to the left / right with equal probability
- The ground-truth values of state A to E are $\frac{1}{6}, \frac{2}{6}, \frac{3}{6}, \frac{4}{6}, \frac{5}{6}$
- Does TD(0) converge faster and better than MC?



Random Walk Example



Bias and Variance Analysis

- Return $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$ is *unbiased* estimate of $v_\pi(S_t)$
- True TD target $R_{t+1} + \gamma v_\pi(S_{t+1})$ is *unbiased* estimate of $v_\pi(S_t)$
- TD target $R_{t+1} + \gamma V(S_{t+1})$ is *biased* estimate of $v_\pi(S_t)$
- TD target is much lower variance than the return:
 - Return depends on *many* random actions, transitions, rewards
 - TD target depends on *one* random action, transition, reward

Biased Estimation of TD methods

$$\begin{aligned}V_{n+1}(s) &= V_n(s) + \alpha[R + V(s') - V_n(s)] \\&= (1 - \alpha)V_n(s) + \alpha(R + V(s')) \\&= (1 - \alpha)[V_{n-1}(s) + \alpha[R + V(s') - V_{n-1}(s)]] + \alpha(R + V(s')) \\&= \dots \\&= (1 - \alpha)^n \underbrace{V_1(s)}_{\text{Biased by } V_1(s)} + \sum_{i=1}^n \alpha(1 - \alpha)^{i-1}(R + V(s'))\end{aligned}$$

Bias and Variance Analysis

- MC has high variance, zero bias
 - Good convergence properties
 - (even with function approximation)
 - Not very sensitive to initial value
 - Very simple to understand and use
- TD has low variance, some bias
 - Usually more efficient than MC
 - TD(0) converges to $v_{\pi}(s)$
 - (but not always with function approximation)
 - More sensitive to initial value

Quick Note: On-Policy and Off-Policy Learning

- On-policy learning: learn v_π and q_π for policy π that executes and explores
- Off-policy learning: learn v_π and q_π for target policy π from experience collected by behavior policy b
- We only need coverage: every action taken under π is also taken, at least occasionally, under b

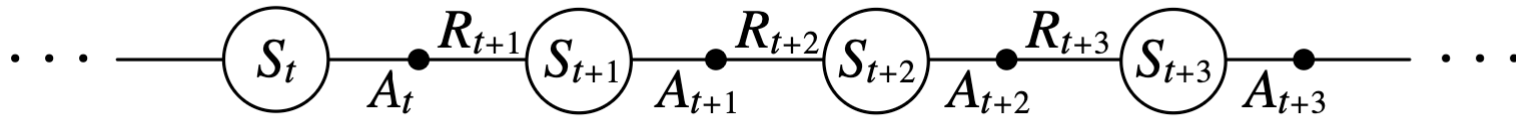
$$\pi(a|s) > 0 \text{ implies } b(a|s) > 0$$

- The goodness of decoupling target and behavior policy:
 - Learn from observing humans or other agents
 - Re-use experience generated from old policies
 - Learn about optimal policy while following exploratory policy
 - Learn about multiple policies while following one policy

Sarsa: On-policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- We can learn an action-value function in a similar manner as a state-value function. Instead of considering transitions from state to state, we now consider transitions from state-action pair to state-action pair



Sarsa: On-policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

Expected Sarsa

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \mathbb{E}_{\pi} [Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] - Q(S_t, A_t) \right] \\ &\leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right], \end{aligned}$$

Eliminate variance due to the
random selection of A_t

Expected Sarsa:

1. Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
2. Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$
3. Loop for each episode:
 4. Initialize S
 5. Loop for each step of episode:
 6. Choose A from S using policy derived from Q (e.g., ε -greedy)
 7. Take action A , observe R, S'
 8. $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S, A)]$
 9. $S \leftarrow S'$
 10. until S is terminal

Q-learning: Off-policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Q-learning: Off-policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Loop for each step of episode:

Choose A from S using policy derived from Q (e.g., ε -greedy)

Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

behavior policy

$S \leftarrow S'$

target policy

until S is terminal

- The learned action-value function approximates q^*
- If all state-action pairs continue to be updated, Q has been shown to converge with probability 1 to q^*

Quick Recap: Temporal-Difference Learning

- Temporal-Difference (TD) methods: combine Monte Carlo methods with Dynamic Programming methods that wait only until the **next time step** and **bootstrap** value functions from existing estimates

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

We call this formulation 1-step TD

We can also have n-step TD

N-step TD Prediction

- n-step TD:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1}R_{t+n} + \gamma^n V(S_{t+n}) - V(S_t)]$$

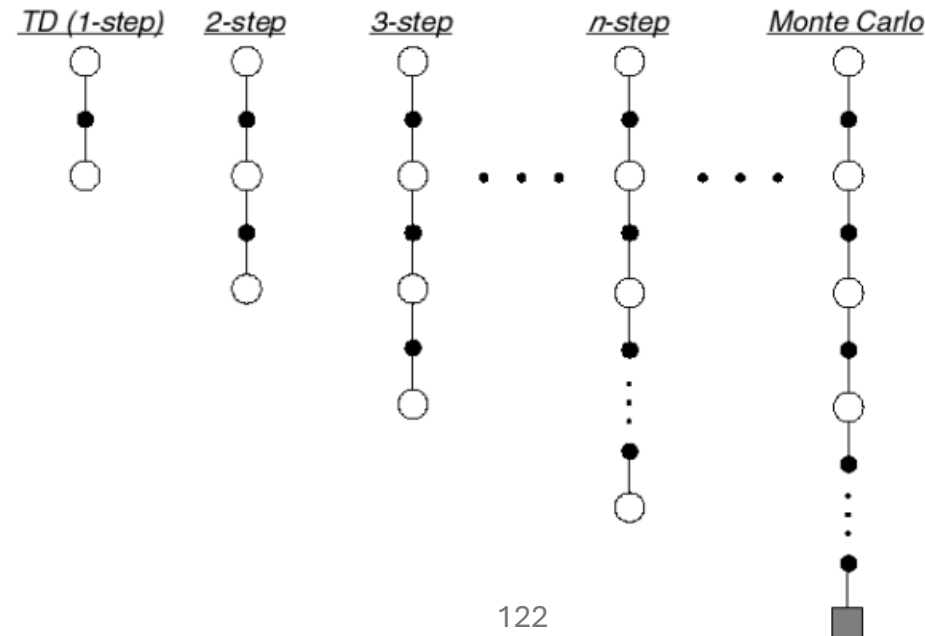
N-step TD Prediction

- n-step TD:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1}R_{t+n} + \gamma^n V(S_{t+n}) - V(S_t)]$$

- When $n \rightarrow \infty$, n-step TD becomes an MC method:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1}R_T - V(S_t)]$$



N-step TD Prediction

n-step TD for estimating $V \approx v_\pi$

Input: a policy π

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$

All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:

Initialize and store $S_0 \neq$ terminal

$T \leftarrow \infty$

No bootstrapping until time
step $t + n$

Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take an action according to $\pi(\cdot | S_t)$

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then $T \leftarrow t + 1$

$\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)

 If $\tau \geq 0$:

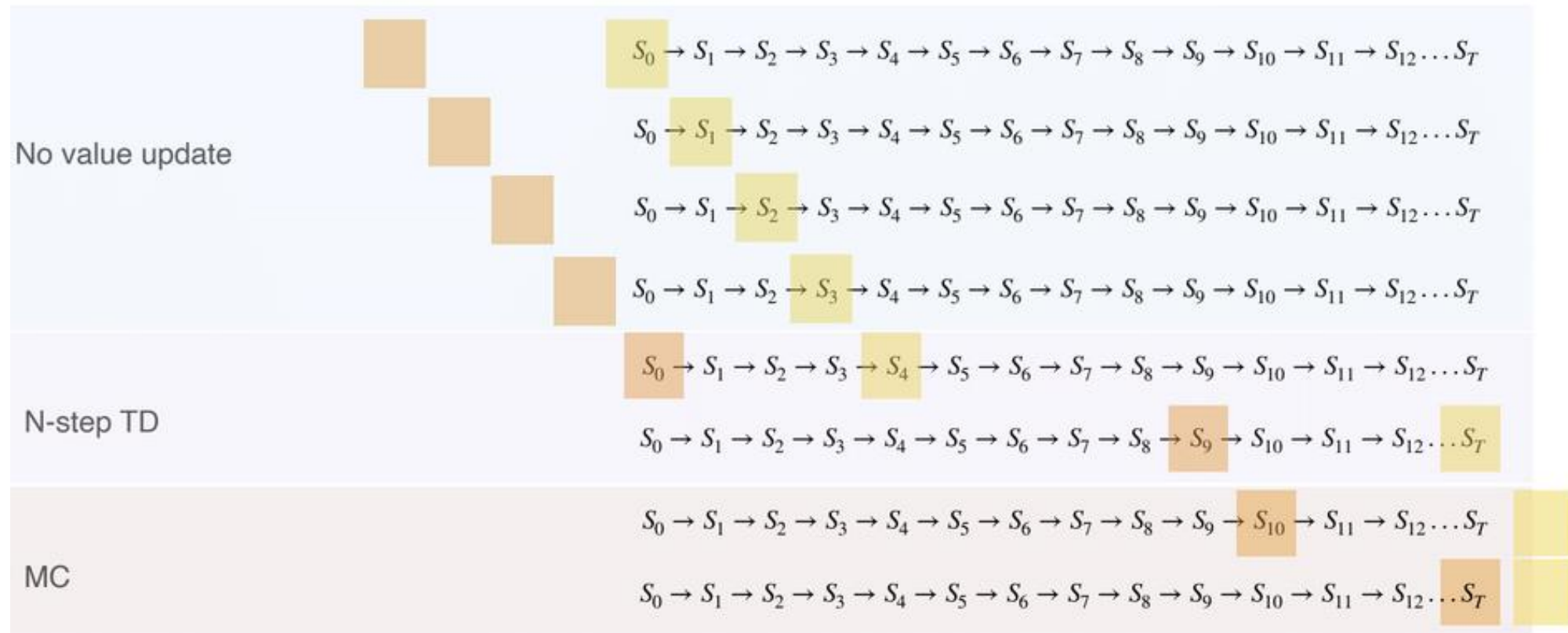
$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ ($G_{\tau:\tau+n}$)

$V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$

Until $\tau = T - 1$

N-step TD Prediction



On-policy n-step Action-Value Methods

- Action-value form of n-step return

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q_{t+n-1}(S_{t+n}, A_{t+n}), \quad n \geq 1, 0 \leq t < T-n,$$

- n-step Sarsa

$$Q_{t+n}(S_t, A_t) \doteq Q_{t+n-1}(S_t, A_t) + \alpha [G_{t:t+n} - Q_{t+n-1}(S_t, A_t)]$$

- n-step expected Sarsa

$$G_t^{(n)} \doteq R_{t+1} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \sum_a \pi(a|S_{t+n}) Q_{t+n-1}(S_{t+n}, a)$$

Off-policy n-step Action-Value Methods

- Importance-sampling ratio

$$\rho_{t:h} \doteq \prod_{k=t}^{\min(h, T-1)} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

- Weighted estimated value functions with importance-sampling ratio
- Off-policy n-step TD

$$V_{t+n}(S_t) \doteq V_{t+n-1}(S_t) + \alpha \rho_{t:t+n-1} [G_{t:t+n} - V_{t+n-1}(S_t)], \quad 0 \leq t < T,$$

- Off-policy n-step Sarsa

$$Q_{t+n}(S_t, A_t) \doteq Q_{t+n-1}(S_t, A_t) + \alpha \rho_{t+1:t+n} [G_{t:t+n} - Q_{t+n-1}(S_t, A_t)]$$